

TEKNİK DOKÜMAN

Yayın Merkezi

Mimari, API ve Ölçekleme Referansı

İçindekiler

Yayın Merkezi — Teknik Doküman

1. Proje Özeti ve Genel Bakış

Projenin amacı

Temel teknoloji yığını

Yetenekler

Sınırlar (bu depo şu an neyi yapamıyor)

Uçtan uca iş akışları — özet

2. Mimari ve Tasarım

Sistem mimarisi

Veri akışı — canlı altyazı örneği (en karmaşık akış)

Kullanılan tasarım kalıpları

Backend çatısı — Quarkus (bir Spring Boot uygulamasından taşındı)

Container'lar — tek tek, neden var

Backend vs video-worker — bayrak tablosu

Donanım kodlayıcılar ve rendition üretimi

DVR kaydı — MediaMTX'in dışında, ayrı bir ffmpeg süreci

HLS + nginx önbellekleme

Triton — KServe v2 protokolü ve iç çalışma mantığı

WebSocket protokolü — gerçek mesaj örneği

Frontend Teknoloji Yığını

Canlı oynatıcı — geri sarma, ekran görüntüsü, klip mimarisi

3. Kurulum ve Başlatma

Gereksinimler

Lokal çalıştırma adımları

Yapılandırma (öne çıkan `.env` alanları)

4. Veri Modeli ve Veritabanı

Varlıklar (özet)

İlişkiler (ER)

Veritabanı geçişleri (Flyway)

MinIO bucket'ları

5. API ve Entegrasyon Dokümantasyonu

Endpoint detayları

Paket paket istek/yanıt referansı

Hata kodları ve formatı

Dış servis entegrasyonları ve kimlik doğrulama

6. Dağıtım ve DevOps

CI/CD

Konteynerizasyon ve sunucu yapısı

Sağlık kontrolleri (`healthcheck`)

İzleme ve loglama

İzleme ve Admin Panel — Derinlemesine

7. Test ve Kalite Güvence

Test stratejisi

Test komutları

8. Ölçeklenme Planı ve Yük Testi

Ölçekleme hedefi ve darboğaz haritası

Ürün kararı: altyazı her zaman mı, yalnızca izlenen mi

İlerleme durumu

Yük testi sonuçları (ilk gerçek koşum)

GPU kapasite sınırı — genel ders

Ölçülen VRAM tüketimi (RTX 4050, 6141 MiB — tarihsel ölçüm)

Batching — göç öncesi ölçüm (eski `stt-worker`, artık kod yok)

Yatay ölçekleme — plan, HENÜZ UYGULANMADI

Diğer belgeler

Yayın Merkezi — Teknik Doküman

Bu, projenin **tek** teknik dökümanı — mimari, kurulum, veri modeli, API, DevOps, test, katkı standartları ve ölçeklenme/vaka analizi geçmişisi hepsi burada. Ayrı konu dosyaları (`teknik-mimari-dokumani.md`, `triton-model-mimarisi.md`, `sistem-loglari-ekrani.md`, `grafana-dashboardlari.md`, `olcekleme-*.md`, `yuk-testi-*.md`, `altyazi-hata-analizi-*.md`, `admin-paneli-*.md`, `analitik-dashboard-plani-*.md`, `batching-boyut-olcumu.md`, `triton-yatay-olcekleme-plani.md`) bu dosyaya taşınıp kaldırıldı — yeni bilgi buraya eklenmeli, yeni bir konu dosyası açılmamalı. Yanında yalnızca `README.md` (hızlı kurulum) ayrı duruyor.

1. Proje Özeti ve Genel Bakış

Projenin amacı

Çok kanallı canlı TV/radyo izleme platformu:

- Canlı yayın izleme (HLS), kanal başına isteğe bağlı kalite (rendition)
- 7 günlük geriye sarma (DVR) ve kayıttan klip çıkarma
- Yüklenen videoların yönetimi ve önizleme klibi üretimi
- Canlı, çok dilli otomatik altyazı** — Whisper (konuşma tanıma) + Marian (çeviri), GPU üzerinde Triton Inference Server ile
- Admin paneli: kullanıcı/rol yönetimi, sistem sağlığı, sistem logları, analitik

Kaynak yayınlar (RTSP/RTMP/SRT/UDP/HLS) MediaMTX'e alınır, oradan HLS olarak izleyicilere dağıtılır. Kimlik doğrulama Keycloak (OIDC) üzerinden.

Temel teknoloji yığını

Katman	Teknoloji
Backend	Java 21, Quarkus 3.37.4 , Hibernate ORM + Panache, RESTEasy Reactive
Veritabanı	PostgreSQL (<code>yayin_merkezi</code>), Flyway migration (V1–V28)
Kimlik	Keycloak (OIDC), realm: <code>YayinYonetimi</code>
Nesne depolama	MinIO (S3 uyumlu) — klip, video, ekran görüntüsü, DVR segmentleri
Ara katman	Redis — pub/sub (altyazı bildirimi) + kuyruk (klip/video işleri)
Medya sunucusu	MediaMTX (RTSP/HLS giriş-çıkış, kayıt)
Yapay zekâ	Triton Inference Server — faster-whisper + Marian, ikisi de CTranslate2 (Marian eskiden ONNX Runtime/ <code>optimum</code> kullanıyordu)
Frontend	React 19, TypeScript, Vite 8, Radix UI + <code>class-variance-authority</code> (shadcn kalıbı), Tailwind CSS 4, <code>hls.js</code>
İzleme	Prometheus + Grafana + Loki/Promtail + dcgm-exporter (GPU)
Konteynerleştirme	Docker Compose (tek <code>docker-compose.yaml</code> , 18 servis)

Önemli mimari not: Backend ve video-worker **aynı jar'**ı çalıştırır; hangi bileşenin (VAD, DVR kaydı, klip işçisi, video işçisi) hangi container'da aktif olacağı ortam değişkeni bayraklarıyla belirlenir (`VAD_ENABLED`, `DVR_RECORDER_ENABLED`, `CLIPS_WORKER_ENABLED`, `VIDEO5_WORKER_ENABLED`). Ayrıntılı bayrak tablosu: \$2.

Yetenekler

- Canlı yayın izleme** — RTSP/RTMP/SRT/UDP/HLS kaynaklardan alıp HLS olarak dağıtma, kanal başına birden fazla rendition (kalite), rendition'lar yalnızca gerçekten izlenirken üretiliyor (\$2 "Donanım kodlayıcılar").
- Radyo** — HLS/RTSP/RTMP/SRT/UDP/WHEP kaynakları doğrudan, Icecast/ Shoutcast (MP3) kaynakları ffmpeg köprüsüyle.

- **DVR (geriye sarma)** — kanal başına açılıp kapatılabilir, sürekli segment kaydı; izleyici canlı akış içinden geriye sarabilir, ayrı bir sayfadan (zaman çizelgesi) da geçmişe gidebilir.
- **Klip çıkarma** — canlı yayından bir zaman aralığını (elle seçilmiş, "kayda başla/durdur" ile işaretlenmiş, ya da önceden planlanmış) MP4 klibe çevirme; klip DVR'ın zaten kaydettiği veriden çıkarılıyor.
- **Video kütüphanesi** — kullanıcı video yükleyip (imzalı URL ile doğrudan MinIO'ya) küçük resim/önizleme klibi otomatik üretiliyor, isteğe bağlı olarak videoya da otomatik altıyazı üretilabiliyor.
- **Canlı çok dilli otomatik altıyazı** — konuşma tanıma (Whisper, İngilizce pivot) + çeviri (Marian, `.env` 'de tanımlı **sınırsız sayıda** hedef dile) — hem canlı yayın hem yüklenen videolar için.
- **Ekran görüntüsü galerisi** — canlı izlerken ya da geriye sarılmış bir andan, tarayıcıda yakalanan kare.
- **Admin paneli** — kullanıcı/rol yönetimi (Keycloak ile), sistem sağlığı, Türkçe'ye çevrilmiş sistem logları, içerik/kullanıcı analitiği.
- **Gözelemlenebilirlik** — Prometheus+Grafana (metrikler), Loki+Promtail (loglar), dcgm-exporter (GPU) — hepsi Docker Compose'un bir parçası.

Sınırlar (bu depo şu an neyi yapamıyor)

- **TLS/HTTPS yok** — `nginx.conf` yalnızca düz HTTP sunuyor, dışarı açmak için ayrı bir reverse proxy/sertifika yönetimi gerekiyor (bu depoya dahil değil).
- **Tek makine, tek GPU varsayımı** — Triton'ı birden fazla GPU'ya/makineye dağıtmak yazılı bir **plan** (§8 "Yatay ölçekleme"), henüz kod/config olarak uygulanmadı.
- **Otomatik yedekleme yok** — Postgres/MinIO için hazır bir yedekleme mekanizması bu depoda tanımlı değil (elle prosedür: `docs/kullanim-kilavuzu.md` §7).
- **CI/CD pipeline yok** — build/test/dağıtım tamamen elle, `docker compose build / up` ile (bkz. §6).
- **Otomatik test kapsamı sınırlı** — yalnızca backend'in bir kısmında (5 test sınıfı) birim testi var; frontend'de ve uçtan uca hiç otomatik test yok (bkz. §7).
- **WebSocket kimlik doğrulaması yok** — canlı altıyazı soketi (`/ws/altıyazı/{channelId}`) kanal ID'sini bilen herkese açık.
- **Otomatik yatay ölçekleme (auto-scaling) yok** — kaç kanal/kaç GPU gerektiği elle hesaplanıp elle yapılandırılıyor, bir orkestrasyon katmanı (Kubernetes vb.) kullanılmıyor.
- **Video/klip altıyazısı ile canlı altıyazı AYNI GPU kapasitesini paylaşıyor** — ayrı bir GPU havuzu/önceliklendirme yok; biri diğerinin gecikmesini etkileyebilir.
- **Coğrafi/CDN dağıtımı yok** — tek origin'den (nginx) yayın dağıtılıyor, bölgesel önbellekleme/CDN entegrasyonu bu depoda yok.

Uçtan uca iş akışları — özet

Aşağıdaki beş akış, kullanıcı açısından sistemin **tamamını** kapsıyor. Her birinin ayrıntılı paket/sınıf düzeyi anlatımı için parantez içindeki bölüme bakın.

1. Bir kanal eklenip yayına girmesi:

```
Yönetici: POST /api/channels (isim, kaynak adresi, path, rendition listesi)
  → ChannelService: Postgres'e satır + MediaMTX API'sine path tanımı
  → MediaMTX kaynağa bağlanır, HLS üretmeye başlar
  → İzleyici: GET /api/channels → hlsUrl → tarayıcı hls.js ile oynatır
```

(§5 "channel" paketi, §2 "Container'lar" → `mediamtx`)

2. İzleyicinin canlı altıyazı görmesi:

```
İzleyici: kanalı açar → SubtitleOverlay.tsx WebSocket'e bağlanır
  (arka planda, HER aktif kanal için sürekli çalışan)
ffmpeg → VAD → Whisper (Triton) → EN pivot → Marian (Triton, N dil paralel)
  → Postgres → Redis pub/sub → WebSocket → tarayıcı
  → SubtitleOverlay: baslangic <= playingDate() < bitis eşlemesi
```

(§2 "Veri akışı" + "Triton" bölümü, §5 "VAD"/"subtitle" paketleri)

3. Bir klip çıkarılması:

Kullanıcı: canlı izlerken "kayda başla" / DVR sayfasında aralık seçip
"klip oluştur" / planlı kayıt zamanı gelir
→ Postgres: clips satırı (BEKLIYOR) → Redis kuyruğu → ClipWorker
→ DVR'dan akış çekilir → MinIO'ya yazılır → status=HAZIR
→ Kullanıcı Klipler sayfasından yoklamayla görür, indirir

(§5 "clip"/"dvr" paketleri)

4. Bir video yüklenip izlenmesi:

Kullanıcı: POST /api/videos → imzalı URL → tarayıcı doğrudan MinIO'ya
PUT eder → POST ../tamamlandi → VideoWorker (küçük resim, önizleme)
→ (opsiyonel) VideoSubtitleWorker (VAD+Triton, WebVTT)
→ Kullanıcı: video kütüphanesinde izler, altyazı seçebilir

(§5 "video" paketi)

5. Bir yöneticinin sorun teşhis etmesi:

Şikayet: "altyazı gelmiyor" → Admin panel → Sistem Sağlığı (7 bileşen)
→ Sistem Logları (Loki → Türkçe yorum) → Grafana (Triton/GPU metrikleri)

(§6 "İzleme ve Admin Panel")

2. Mimari ve Tasarım

Sistem mimarisi

Servis odaklı, event/queue destekli monolit — mikroservis değil: tek bir Quarkus uygulaması (jar) iki farklı rolde (backend / video-worker) iki ayrı container'da çalışıyor, aralarında **doğrudan çağrı yok**, yalnızca Redis ve Postgres üzerinden haberleşiyorlar. Triton ayrı, bağımsız bir çıkarım servisi (Java'dan HTTP/KServe v2 protokolüyle çağrılıyor). Backend/video-worker ayrıca Keycloak, MediaMTX, MinIO, Prometheus ve Loki'ye de **doğrudan HTTP** ile bağlanıyor — "tek bağ Redis/Postgres" kuralı yalnızca backend ↔ video-worker ikilisi için geçerli, tüm sistem için değil.



backend : REST API + WebSocket + kullanıcı/rol/klip yönetimi + admin panel uçları

Postgres : TEK doğruluk kaynağı – Redis yalnızca bildirim taşıy, kaybolursa yoklama toparlar

18 Docker Compose servisi: postgres, keycloak-postgres, keycloak, minio, redis, mediamtx, backend, triton, video-worker, frontend, dcgm-exporter, postgres-exporter, keycloak-postgres-exporter, redis-exporter, prometheus, loki, promtail, grafana.

Veri akışı — canlı altyazı örneği (en karmaşık akış)

```
ffmpeg (RTSP'den ses çeker)
  → Silero VAD (konuşma tespiti, video-worker içinde)
  → ses bölütü (segment)
  → Java kuyruğu (ArrayBlockingQueue, sınırlı)
  → Triton: Whisper (task=translate, EN pivot metin üretir)
  → her hedef dil için paralel: Triton Marian (EN → X çeviri)
  → Postgres (altyazı satırı kalıcı kayıt)
  → Redis pub/sub (altyazı:<kanalId> kanalına yayın)
  → backend WebSocket (/ws/altyazı/{channelId})
  → tarayıcı (SubtitleOverlay.tsx, mutlak zaman damgasıyla eşleme)
```

Kritik kural: `SubtitleOverlay.tsx` altyazıyı **mutlak zaman damgasıyla** eşliyor: `başlangic <= playingDate() < bitis`. Geç kalan altyazı geç gösterilmez, **hiç** gösterilmez — hiçbir yerde hata çıkmaz, ekran boş kalır, loglar temiz görünür. Ölçülen üretim gecikmesi: p50 ~13 sn, p95 ~23 sn (CPU, 8 çekirdek, `small` model, int8) — bu yüzden `ALTYAZI_HLS_GERIDE` bütçesinin p95'i karşılaması gerekiyor (bkz. §3 "Yapılandırma").

Kullanılan tasarım kalıpları

- Repository/Active Record karışımı** — Panache entity'leri (`Channel` , `Clip` , `Subtitle` vb.) hem veri hem sorgu taşıyor, ayrı bir repository katmanı yok (Panache'in kendi felsefesi).
- Merkezi hata modeli** — tüm iş kuralı hataları tek bir `AppException` (factory metotlu: `AppException.notFound(...)` , `.conflict(...)` vb.) üzerinden, `ErrorCode` enum'u HTTP durum koduna eşleniyor, `AppExceptionHandler` / `ValidationExceptionHandler` / `GenericExceptionHandler` üç ayrı `ExceptionHandler` ile tek tip `ErrorResponse` JSON'ına dönüştürülüyor (bkz. §5).
- Bayrakla rol ayrımı** — aynı kod tabanı, ortam değişkeniyle farklı sorumluluk üstleniyor (aşağıdaki backend/video-worker bayrak tablosu).
- Şablon + build-time kod üretimi** — Triton'daki Marian modelleri artık elle yazılmış config değil, `triton/templates/` altındaki TEK şablondan (`export_models.py` aracılığıyla) `.env` 'e göre üretiliyor; yeni dil eklemek kod değişikliği gerektirmiyor (bkz. `triton/export_models.py` başındaki "TAM DINAMİK MARIAN" notu). Java tarafında da `STT_TARGET_LANGS` tek kaynak: `VadService` (canlı kanal) ve `VideoSubtitleWorker` (yüklenen video) hedef dil → model eşlemesini `stt.target-langs` 'tan aynı şekilde üretiyor — ikisi de eskiden ayrı ayrı `tr/de/ru` sabit kodluyordu, ikisi de düzeltildi.
- Doğruluk kaynağı hep veritabanı** — Redis yalnızca bildirim/kuyruk taşıyor; kaybolursa periyodik yoklama (`@Scheduled`) durumu toparlıyor (örn. `VadService.sync()` , her 30 saniyede kanal/işçi eşitlemesi).

Backend çatısı — Quarkus (bir Spring Boot uygulamasından taşındı)

Neden Quarkus: hızlı başlangıç süresi, düşük bellek ayak izi (konteyner ortamı için önemli), Panache ile az kod yazarak veritabanı erişimi, Keycloak/Redis/MinIO'ya hazır entegrasyon eklentileri.

Önemli mimari not — bu bir taşıma, sıfırdan seçim değil: `AppException` , `ErrorCode` ve üç `ExceptionHandler` sınıfının javadoc'ları açıkça bir **Spring Boot uygulamasından Quarkus'a taşındığını** belirtiyor — iş mantığı karakter karakter aynı kalmış, yalnızca framework'e özel katman değiştirilmiş (`HttpStatus` → `Response.Status` , `@ExceptionHandler` → `ExceptionHandler` , `@RestControllerAdvice` → `@Provider`). `AppException` 'ın kendi javadoc'u: *"Framework'e hiç bağımlı değil — Spring'den Quarkus'a geçişte bu dosya karakter karakter aynı kalabilir."* Bu desen 6 dosyada tekrarlanıyor. Geçişin **kendisinin gerekçesi** (neden Spring'den ayrıldığı) kod içinde belgelenmemiş — yalnızca Quarkus'un genel avantajları (yukarıda) not düşülmüş.

Maven koordinatları: `quarkus-bom` 3.37.4, Java 21 (`maven.compiler.release`).

Bağımlılık tablosu — her birinin nedeni:

Bağımlılık	Ne işe yarar	Neden seçildi / not
<code>quarkus-rest</code> + <code>quarkus-rest-jackson</code>	REST uç noktaları + JSON	Quarkus'un yeni nesil REST katmanı (eski <code>quarkus-resteasy</code> değil)

Bağımlılık	Ne işe yarar	Neden seçildi / not
<code>quarkus-hibernate-orm-panache</code> + <code>quarkus-hibernate-orm</code>	ORM	Panache, Active Record tarzı katman ekleyip repository boilerplate'ini azaltıyor
<code>quarkus-jdbc-postgresql</code>	PostgreSQL sürücüsü	—
<code>quarkus-flyway</code>	Şema migration (V1-V28)	<code>quarkus.hibernate-orm.database.generation=validate</code> — Hibernate şemayı asla değiştirmiyor , yalnızca doğruluyor, tüm şema değişikliği Flyway'den geçmek zorunda
<code>quarkus-keycloak-authorization</code>	Gelen isteklerde OIDC token doğrulama + rol kontrolü	<code>role-claim-path</code> BİLİNÇLİ OLARAK ayarlanmıyor — client adını ikinci kez gömmek için (boş bırakılınca Quarkus hem <code>realm_access/roles</code> hem <code>resource_access/{client}/roles</code> yollarına otomatik bakıyor)
<code>quarkus-keycloak-admin-rest-client</code>	Keycloak admin API (kullanıcı CRUD)	Admin panelin kullanıcı yönetimi; client'in service account'unda <code>manage-users</code> / <code>view-users</code> / <code>query-users</code> / <code>view-realm</code> rolleri gerekiyor
<code>quarkus-rest-client</code> + <code>-jackson</code>	Dış servise (MediaMTX) tip güvenli istemci	<code>MediaMtxClient</code>
<code>quarkus-websockets</code> + <code>quarkus-websockets-client</code>	Canlı altyazı akışı	Kod klasik <code>jakarta.websocket</code> (<code>@ServerEndpoint</code>) API'sini kullanıyor, Quarkus'un daha yeni "WebSockets Next" API'si değil — gerekçe kodda belirtilmemiş (gözlemsel)
<code>quarkus-redis-client</code> + <code>quarkus-redis-cache</code>	Redis (pub/sub + kuyruk)	—
<code>quarkus-scheduler</code>	Periyodik arka plan işleri	Klip kuyruğu yoklama, retention temizliği
<code>quarkus-hibernate-validator</code>	DTO alan doğrulama (<code>@NotNull</code> vb.)	<code>ConstraintViolationException</code> → <code>ValidationExceptionMapper</code>
<code>quarkus-smallrye-openapi</code>	Swagger UI + OpenAPI JSON	<code>/docs</code> , <code>/docs/openapi.json</code>
<code>quarkus-logging-json</code>	JSON log formatı	Loki/Promtail'in yapılandırılmış log ayrıştırması için
<code>quarkus-micrometer-registry-prometheus</code>	<code>/q/metrics</code>	Kuyruk derinliği/düşme gibi özel metrikler buradan
<code>io.minio:minio</code> 8.6.0 (düz SDK)	MinIO/S3 istemcisi	<code>quarkus-minio</code> eklentisi Quarkus 3.37 ile build-step hatası verdiği için (3.8.x config hatası, 3.9.x <code>NoSuchMethodError</code>) düz SDK + elle üretilen bean tercih edildi
<code>com.squareup.okhttp3:okhttp-jvm</code> 5.1.0	MinIO SDK'nın HTTP bağımlılığı	okhttp 5.x Kotlin Multiplatform yayınlanıyor — düz <code>okhttp</code> artifact'i yalnızca variant metadata taşıyor, JVM sınıfları için <code>-jvm</code> varyantı AÇIKÇA eklenmezse <code>cannot access okhttp3.HttpUrl</code> ile derleme patlıyor
<code>com.microsoft.onnxruntime:onnxruntime</code> 1.20.0	Java'da ONNX modeli çalıştırma	Marian için DEĞİL (Triton tarafı ayrı, Python/CTranslate2) — <code>SileroVad.java</code> 'daki VAD (konuşma tespiti) modelini doğrudan Java sürecinde, ağız çalıştırmak için
<code>quarkus-junit</code> + <code>rest-assured</code> (test)	Test çatısı	—

Merkezi hata modeli — mimari: Tek akış: `AppException` (factory metotlu, örn. `AppException.notFound(...)`) → `ErrorCode` enum'u (her değer bir `Response.Status` 'a sabit eşli) → uç `@Provider` `ExceptionHandler` : `AppExceptionHandler` (bilinen iş kuralı hataları), `ValidationExceptionHandler` (`@Valid` DTO doğrulama, alan bazlı `FieldError` listesi üretiyor),

`GenericExceptionHandler` (geri kalan HER ŞEY — istemciye asla ham stack trace gitmiyor, yalnızca loglanıp genel 500 dönüyor). Üçü de aynı `ErrorResponse` JSON şekline yakınsıyor. Yeni bir hata durumu eklemek = `ErrorCode` 'a bir değer + `AppException` 'a bir factory metod; mapper'lara asla dokunulmuyor.

WebSocket — neden yoklama değil: `SubtitleSocket.java` 'nın javadoc'unda belgelenen gerekçe: "Yoklama saniyede bir istek demektir ve alt yazı yine de bir tik geç görünüyordu. WebSocket'te alt yazı üretilir üretilmez gidiyor." Bilinen eksik: **kimlik doğrulaması yok**, bilinçli bırakılmış çünkü HLS yayınının kendisi de korumasız — ikisi birlikte çözülmesi gereken bir sorun olarak not düşülmüş.

Aynı jar, iki imaj — `Dockerfile.jvm` vs `Dockerfile.worker`: Her ikisi de `eclipse-temurin:21-jre-noble` tabanlı — UBI9'dan BİLİNÇLİ OLARAK taşınmış (UBI9'un `microdnf` 'i `ffmpeg`'i kuramıyordu: "No package matches 'ffmpeg-free'", EPEL eklemek "UBI üzerinde desteklenmiyor ve kırılabilir" bulunmuş). İkisinin aynı Ubuntu tabanında olması ayrıca "işçide çalışıyor backend'de çalışmıyor" türü `ffmpeg` sürüm farkı sorunlarını kapatıyor.

	<code>Dockerfile.jvm</code> (backend)	<code>Dockerfile.worker</code> (video-worker)
Ekstra paket	yalnızca <code>ffmpeg</code>	<code>ffmpeg</code> + <code>libva2</code> + <code>libva-drm2</code> (VA-API çalışma zamanı)
Neden fark	Backend yalnızca <code>-c copy</code> ile remux yapıyor (DVR replay, klip kırma) — gerçek KODLAMA yok	Video-worker gerçek donanım kodlama yapıyor (thumbnail, önizleme klibi, faststart remux)
Unix kullanıcısı	<code>backend</code> (uid 1000)	<code>isci</code> (uid 1000)
Port	8081	8082 (HTTP sunmuyor ama Quarkus yine de bir port açıyor, çakışmasın diye farklı)
Aygıt erişimi	yok	<code>/dev/dri</code> (VA-API için, compose'da <code>group_add: video</code>)

Neden ayrı bir servis/dil değil, aynı jar: Veritabanı, MinIO ve Redis istemcileri, entity'ler ve durum makinesi zaten yazılı — ikinci bir dilde yeniden yazmak iki ayrı doğruluk kaynağı demek olurdu. Hangi işlerin hangi konteynerde çalışacağı tamamen ortam değişkeni bayraklarıyla belirleniyor (aşağıdaki tablo).

Java paket yapısı — özellik bazlı (feature-based), katman bazlı DEĞİL: `src/main/java/org/example/` altında `controllers/`, `services/`, `repositories/` gibi teknik katman paketleri yok. Her iş alanı kendi paketini taşıyor: `auth`, `channel`, `clip`, `dvr`, `etkinlik`, `exception`, `media`, `playback`, `radio`, `screenshot`, `sistemlog`, `storage`, `subtitle`, `user`, `VAD`, `video`, `viewer`. Örnek — `channel/` paketinin içi:

channel/	
ChannelResource.java	(REST uç noktası)
ChannelService.java	(iş mantığı)
ChannelDeletionService.java	
ChannelRestorer.java	
MediaMtxClient.java	(dış servis istemcisi)
MediaMtxService.java	
HlsPlaylist.java, Rendition.java, SourceProbe.java, TranscodeCommand.java	
dto/	(istek/yanıt DTO'ları)
entity/Channel.java	(Panache entity)

Her özellik kendi Resource+Service+Entity+DTO üçlüsünü birlikte taşıyor — bir özelliği anlamak için tek dizine bakmak yeterli.

Container'lar — tek tek, neden var

`postgres` — birincil veritabanı

`yayin_merkezi` DB, `app_user` kullanıcı. Kanallar, radyolar, kullanıcıların (Keycloak'takinin yerel aynası) satırları, alt yazılar, videolar, klipler, planlı kayıtlar, ekran görüntüleri, etkinlik denetim izi burada. **Neden Postgres** (MySQL vb. değil) kodda/dökümanlarda açık bir gerekçe yok — gözlemsel: proje baştan Postgres üzerine kurulmuş.

```
postgres:
  image: postgres:16
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-app_user} -d ${POSTGRES_DB}"]
```

keycloak-postgres neden ayrı bir instance: Keycloak kendi şema geçişlerini kendi yönetiyor, uygulamanın Flyway'yle asla karışmıyor — her ikisi de aynı `postgres:16` imajından ama tamamen bağımsız iki konteyner/veri kümesi.

keycloak — kimlik doğrulama

Parola hashleme, brute-force koruması, token imzalama gibi güvenlik-kritik işler kanıtlanmış bir ürüne bırakılıyor, kendi auth kodu yazılmıyor. `realm-export.json` client secret + ilk admin kullanıcıyı (`admin1` / `12345678`) taşıyor, ilk açılışta otomatik yükleniyor.

redis — bildirim kanalı ve iş kuyruğu (doğruluk kaynağı DEĞİL)

Kod tabanının açık ilkesi (`ClipQueue.java` javadoc'undan birebir): *"Redis burada doğruluk kaynağı değil, bildirim kanalıdır. İşin kalıcı hali `clips` tablosunda durur; Redis yalnızca 'yeni iş var' haberini taşır. [...] Bu ayrım sayesinde Redis tamamen çöke bile hiçbir iş kaybolmaz — yalnızca gecikme, süpürücünün aralığına düşer."* Aynı ilke `ClipQueue.publish()` 'te pratiğe dökülmüş: Redis'e yazma başarısız olursa **istisna fırlatılmıyor**, iş zaten Postgres'te **BEKLIYOR** durumunda duruyor ve bir süpürücü onu bulacak.

`quarkus.redis.devservices.enabled=false` **açıkça kapatılmış** — gerekçe: *"Yazılmazsa Dev Services kendi Redis konteynerini rastgele bir portta açar ve compose'daki Redis'i yok sayar; klip bildirimleri her yeniden başlatmada kaybolan geçici bir örneğe gider."*

BLMOVE neden BRPOP değil (`ClipQueue.java`): iki liste var — `bekleyen` ve `isleniyor` . **BLMOVE** işi tek atomik adımda bir listeden diğerine taşıyor. Gerekçe: *"Basit bir **BRPOP** kullanılsaydı, işçi işi aldıktan hemen sonra çökerse iş hiçbir listede olmaz ve Redis tarafında iz bırakmadan kaybolurdu."* Aynı desen `VideoQueue` , `VideoSubtitleQueue` 'da da birebir tekrarlanıyor. RabbitMQ/Kafka'nın neden değerlendirilmediğine dair kodda bir not yok — gözlemsel: Postgres zaten doğruluk kaynağı olduğu için Redis'e yalnızca hafif, kaybı tolere edilebilir bir bildirim katmanı gerekiyordu.

Pub/sub için ayrı istemci gerekiyor (`application.properties` , `SubtitleBroadcaster.java` , `DvrSignal.java` 'da birebir aynı yorum tekrarlanıyor):

```
PUB/SUB ICIN AYRI ISTEMCI. Varsayılan havuzu klip kuyruğu kullanıyor ve
BLMOVE her isciyi 2 saniyeye kadar bir baglantida tutuyor. Abonelik aynı
havuzdan baglanti isteyince cakisiyordu; OLCULDU: subscribeToPattern 41
SANIYE bloke kaldı ve donene kadar hicbir altyazi tarayiciya ulasmadi.
```

Çözüm: `@RedisClientName("pubsub")` ile `SubtitleBroadcaster` ve `DvrSignal` 'e tamamen ayrı bir Redis istemcisi/bağlantı havuzu tanımlanmış.

minio — nesne depolama (S3 uyumlu)

Neden MinIO (doğrudan disk değil): `VideoStorage.java` 'nın sınıf javadoc'unda açık: *"Dosya backend'den geçmiyor. Yükleme imzalı PUT adresiyle doğrudan tarayıcıdan, indirme imzalı GET adresiyle doğrudan MinIO'dan."* S3 API'sinin sunduğu **presigned URL** mekanizması bilinçli bir tasarım tercihi: büyük dosya baytları backend sürecinin CPU/bellek/bant genişliğinden hiç geçmiyor.

İmzalı URL mekanizması — iki ayrı `MinioClient` üretiliyor (`MinioClientProducer.java`):

```
@Produces @ApplicationScoped
MinioClient minioClient() { ... } // ic ag: http://minio:9000

@Produces @ApplicationScoped @PresignClient
MinioClient presignClient() { ... } // TARAYICININ erisebilecegi adres
```

Gerekçe (aynı dosyanın javadoc'u): *"Ayrı olması şart: backend compose ağındayken `minio.url` `http://minio:9000` olur, ama imzalı indirme adresi tarayıcıda açılacak ve tarayıcı o ismi çözemez. Adresi sonradan değiştirmek de mümkün değil — S3 v4 imzası Host başlığını da imzalıyor, host'u elle değiştirmek imzayı geçersiz kılar."* `@PresignClient` özel bir `@Qualifier` — CDI'da `@Named` tek başına `@Default` 'u kaldırmadığı için iki üretici varsayılan kalıp `AmbiguousResolutionException` fırlatıyordu.

Bucket yapısı:

Bucket (varsayılan)	Env değişkeni	Kullanıcı sınıfı
videolar	VIDEO0S_BUCKET	VideoStorage
klipler	CLIPS_BUCKET	ClipStorage
ekran-goruntuleri	SCREENSHOTS_BUCKET	ScreenshotStorage
dvr	DVR_BUCKET	DvrStorage

Video ve klip için ayrı kova olmasının gerekçesi (`VideoStorage.java`): "Klipten ayrı bir sınıf ve ayrı bir kova: klip TTL'i indirme için kısa (15 dk) tutulmuş, oysa video oynatmada aynı süre videonun ortasında kopmaya yol açar."

Retention politikası bucket seviyesinde bir S3 lifecycle kuralı DEĞİL, uygulama seviyesinde `RetentionSweeper.java` (zamanlanmış iş). Bilinçli varsayılan: "Varsayılan olarak kullanıcı verisi silinmiyor. Klip ve ekran görüntüsü kullanıcının kendi arşivi; zamana bağlı silmek 'arşivim duruyor' beklentisini bozar. Baskıyı kota kursun, saat değil. Silinmesi varsayılan olan tek şey **başarısız** klipler: dosyaları zaten yok, yalnızca kullanıcı sebebini görsün diye bekletiliyorlar."

Neden `quarkus-minio` eklentisi kullanılmıyor (`MinioClientProducer.java`): "Quarkiverse'ün `quarkus-minio` eklentisi kullanılmıyor: 3.8.x sürümü Quarkus 3.37 ile build-step yapılandırma hatası veriyor, 3.9.x ise SDK ile `NoSuchMethodError` üretiyor. Eklenti ayrıca Dev Services ile kendi MinIO konteynerini açıp yapılandırılmış adresi eziyordu."

Üç servisin iş bölümü — tek cümlede: kalıcı/otoriter veri Postgres'te, büyük ikili veri MinIO'da (backend'den hiç geçmeden), Redis yalnızca geçici/kayıp tolere edilebilir sinyal taşıyor — Redis çökerse veri kaybolmaz, yalnızca bildirim gecikir ve bir süpürücü/yoklama devreye girer.

mediamtx — medya sunucusu

Neden MediaMTX: RTSP/RTMP/SRT/HLS/WebRTC gibi birden fazla protokolü **tek ikili dosyada** destekleyen, hafif, açık kaynak bir medya sunucusu — kendi REST API'siyle (path ekleme/silme/listeleme) yönetiliyor (`MediaMtxClient`). Kanallar `mediamtx.yml` 'e elle yazılıyor: dosya kasıtlı olarak `paths: {}` boş bırakılıyor, backend açılışta Postgres'teki `channels` tablosundan okuyup path'leri API üzerinden yeniden kuruyor — **kalıcı kayıt veritabanı**, MediaMTX yalnızca çalışma zamanı durumu.

Neden özel imaj (`Dockerfile.mediatrix`): Resmi `bluenvion/mediatrix` imajı `scratch` tabanlı — içinde kabuk bile yok. Rendition kodlaması MediaMTX'in `runOnDemand` kancasıyla komutu **konteynerin içinde** çalıştırıyor; bu yüzden ffmpeg'in oraya girmesi gerekiyor. Yan fayda: imajın içine konan VAAPI sürücüsü host'takinden daha iyi — host sürücüsü yalnızca CQP (sabit kalite) destekliyordu, imajdaki Intel iHD sürücüsü CBR/VBR açıyor, yani bit hızı hedeflenebiliyor ve DVR disk hesabı öngörülebilir kalıyor.

MediaMTX'in kendi yapılandırma kararları (`mediamtx.yml`):

- `hlsDirectory` verilmiyor → segmentler yalnızca **bellekte**, diske hiç yazılmıyor.
- `hlsMuxerCloseAfter: 10s` (varsayılan 60s'ten düşürüldü) — **ölçüldü:** master playlist'e her yeni istek (her `hls.js` yeniden bağlanması) ayrı bir HLS oturumu açıyor, eskisi hemen kapanmıyor; 5 kez aç/kapa döngüsünde reader sayısı gerçek izleyici sayısı yerine 5'e kadar çıkıyordu.
- `playback: no` — DVR kaydı MediaMTX'in dışına taşınınca (aşağı bkz.) okuyacağı izin kalmadı.
- `pathDefaults.record: no` — **bilinen sınır: MediaMTX S3'ü desteklemiyor** (1.19.3'te hiç yok), yalnızca yerel dosya sistemine yazabiliyor.
- `authInternalUsers` — yayın alma/izleme herkese açık (`user: any`), yönetim API'si yalnızca `127.0.0.1` ve Docker bridge ağından (`172.16.0.0/12`) erişilebilir; **prod notu koddan açıkça yazılı:** bu ağa erişebilen her şey kanal ekleyip silebilir, üretimde gerçek user/pass tanımlanmalı.

frontend — statik sunum + reverse proxy

Statik dosyaları sunar VE backend + MediaMTX'e reverse proxy yapar (`nginx.conf`), tarayıcı tek origin'den (`:3000`) konuşup CORS karmaşası çıkmıyor. Ayrıntı: aşağıdaki "Frontend Teknoloji Yığını" bölümü.

İzleme yığını rolleri

`prometheus` tüm `/metrics` uçlarını tarar; `loki` 7 gün log saklıyor — denetim arşivi değil, operasyonel görünürlük aracı; `promtail` **tek** `docker.sock` 'a dokunan bileşen; `dcgm-exporter` / `postgres-exporter` / `redis-exporter` ilgili servisin iç istatistiklerini Prometheus formatına çevirir. Ayrıntı: \$6 "İzleme ve loglama".

Backend vs video-worker — bayrak tablosu

Bayrak	backend	video-worker	Ne yapar
<code>VIDEOS_WORKER_ENABLED</code>	<code>false</code>	<code>true</code>	Yüklenen video işleme kuyruğunu kim tüketiyor
<code>CLIPS_WORKER_ENABLED</code>	<code>true</code>	<code>false</code>	Klip kuyruğu — ffmpeg gerektirmediği için backend'de kalabiliyor
<code>DVR_RECORDER_ENABLED</code>	<code>false</code>	<code>true</code>	Segment kaydı — ffmpeg gerektirir, backend imajında yok
<code>VAD_ENABLED</code> / <code>VAD_STT_ENABLED</code>	yok	<code>true</code>	VAD + Triton gönderimi — ffmpeg gerektirir
<code>QUARKUS_FLYWAY_MIGRATE_AT_START</code>	varsayılan <code>true</code>	<code>"false"</code>	Migration'ı kim çalıştırıyor — ikisi aynı anda yarış durumu yarattı

Her çiftin **tam olarak birinde** `true`, **diğerinde** `false` olması şart — bu proje boyunca birkaç kez unutulup her segmentin/işin **iki kez** işlenmesine yol açan gerçek bir hata sınıfı. Ayrım, yukarıdaki `Dockerfile.jvm` / `Dockerfile.worker` tablosundaki ffmpeg/VAAPI farkına dayanıyor — yalnızca `video-worker` 'da donanım kodlama kütüphaneleri var.

Donanım kodlayıcılar ve rendition üretimi

`CHANNELS_ENCODER` / `VIDEOS_ENCODER` → `VideoEncoder` enum'u (`NVENC` | `VAAPI` | `YAZILIM`), **iki ayrı yerde ayrı ayrı** ayarlanıyor (canlı yayın rendition'ları `mediamtx` konteynerinde, kütüphane küçük resim/önizleme `video-worker` konteynerinde) çünkü iki konteyner farklı aygıtlara erişiyor ve backend hiçbirini göremediği için seçim **elle** yapılandırmadan geliyor.

Ölçülen maliyet (canlı yayın, 1080p kaynak, rendition başına): VAAPI ~%34 CPU, `YAZILIM` bunun birkaç katı. Karşılaştırma için MediaMTX'in kendisi 12 kanal + 15 radyoyu %25 CPU ile taşıyor — **ölçekleme duvarı kanal sayısı değil, transcode**.

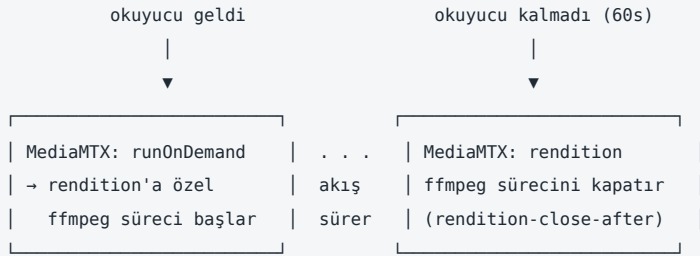
Seçenek	Nasıl çalışıyor	Not
<code>NVENC</code>	Tam GPU hattı: çözme NVDEC, ölçekleme <code>scale_cuda</code> (CUDA'da), kodlama NVENC — kareler hiç sistem belleğine inmiyor. <code>-preset p4</code> (p1 en hızlı, p7 en kaliteli)	<code>nvidia-container-toolkit</code> + compose'da GPU ayrılmış olmalı
<code>VAAPI</code>	<code>/dev/dri</code> üzerinden Intel/AMD donanım kodlama	Bu projede ölçekleme bilinçli olarak yazılımda — tam VAAPI hattı (<code>-hwaccel vaapi</code> ile çözme) denendi, bu donanımda çalışmadı (kod yorumunda açıkça belirtilmiş, gerçek deneme)
<code>YAZILIM</code>	<code>libx264</code> , <code>-preset veryfast -tune zerolatency</code>	Canlı yayında pahalı, birkaç kanaldan fazlasında makineyi doyurur — yalnızca önizleme klipleri için yeterli

Dikkat edilmesi gereken bayraklar (kod yorumlarında belgeli):

- `-map 0:v:0 -map 0:a:0` zorunlu: açıkça seçilmezse filtre yanlış akışa uygulanıp "Error while processing the decoded data" ile düşüyor.
- Küçük resim yakalarken `hwaccel_output_format` **bilerek verilmiyor** — verilseydi kare GPU'da kalır, JPEG'i yazan `mjpeg` kodlayıcı yazılımda olduğu için `hwdownload` zorunlu hale geldi; çözme yine GPU'da yapılıyor (asıl kazanç orada), kare sistem belleğine iniyor.
- `-c:a copy` her rendition'da sabit: ses hiçbir seçenekte yeniden kodlanmıyor (gereksiz CPU/kalite kaybı).

Rendition'lar — talebe bağlı üretim (`TranscodeCommand`): Her rendition **kendi bağımsız** `runOnDemand` kancasına sahip — path'ine ilk okuyucu geldiğinde MediaMTX kendisi ffmpeg sürecini başlatıyor, `channels.rendition-close-after` (varsayılan 60s) süresi boyunca okuyucu kalmazsa yine MediaMTX kendisi kapatıyor. **Kaynak path buna dahil değil** — DVR/kayıt bağımlılığı nedeniyle her zaman canlı kalmaya devam ediyor. **Neden rendition başına bağımsız süreç** (paylaşımlı "tek decode + N encode"

modelinden bilinçli farklı bir tasarım): bedel aynı kanalın 2 rendition'ı eşzamanlı izlenirse kaynağın 2 kez çözülmesi; kazanç, MediaMTX'in path-başına native `runOnDemand` ilkesinin **doğrudan** kullanılabilmesi — paylaşımli tek süreç modelinde iki rendition'a aynı anda ilk izleyici gelirse aynı süreç iki kez başlatılmaya çalışılırdı. **Asıl kazanç: kimse izlemiyorsa süreç hiç çalışmıyor.**



Kaynak path bu döngünün DIŞINDA – DVR/kayıt bağımlılığı nedeniyle okuyucu olsun olmasın her zaman canlı kalır.

Kayıt her zaman kaynak path'ine yazılıyor, rendition'a değil: Bir rendition'a yazmak disk tasarrufu sağlasa da iki bedeli var: (1) kalite kaybı — kaynak 1080p verse bile kayıt rendition'ın çözünürlüğünde kalırdı; (2) rendition talebe bağlı üretildiği için (yukarıya bkz.) o an hiçbir izleyici yoksa üreten ffmpeg süreci hiç çalışmıyor olabilir — kayıt kaynağa bağlanınca bu bağımlılık ortadan kalkıyor, kayıt izleyici sayısından bağımsız her zaman veri alıyor.

DVR kaydı — MediaMTX'in dışında, ayrı bir ffmpeg süreci

MediaMTX S3'ü desteklemediği için DVR kaydı **ayrıca**, `ChannelDvrRecorder` tarafından RTSP'den doğrudan çekilip MinIO'ya aktıtılıyor:

```
ffmpeg -v warning -rtsp_transport tcp -i rtsp://mediamtx:8554/<path> \
-c copy -f mpegts -
```

Bayrak	Gerekçe
<code>-c copy</code>	Yeniden kodlama yok — kaynak hangi kalitedeyse o kaydediliyor
<code>-rtsp_transport tcp</code>	UDP'de paket kaybı kayda kalıcı bozulma olarak işlenir
<code>-f mpegts -</code>	Rastgele sınırdan kesilebilen tek biçim (aşağıya bkz.)
<code>-v warning, error</code> DEĞİL	Yeniden bağlanma/zaman aşımı/paket kaybı UYARI seviyesinde bildiriliyor; <code>error</code> ile stderr boş kalıp ffmpeg beklenmedik bittiğinde tek ipucu kaybolurdu

SegmentStream — sürekli akışı segmentlere bölen sınıf: ffmpeg kesintisiz bir MPEG-TS akışı üretiyor, MinIO SDK ise başı-sonu olan bir `InputStream` bekliyor; bu sınıf akışı segment süresi dolduğunda "bitmiş gibi" gösteriyor, SDK yüklemeyi tamamlıyor, alttaki boru açık kalıp bir sonraki segment kaldığı yerden devam ediyor.

```
ffmpeg — sürekli, sınırsız MPEG-TS akışı
|
▼
SegmentStream — segment süresi dolunca akışı "bitmiş" gösterir
|
(kesim her zaman 188 baytlık paket sınırında)
▼
MinIO SDK — bu segmenti ayrı bir nesne olarak PUT eder
|
▼
alttaki boru AÇIK KALIR — sonraki segment kaldığı yerden devam eder
```

- Neden 188 baytlık hizalama:** MPEG-TS sabit 188 baytlık paketlerden oluşuyor, her paket `0x47` ile başlıyor. Kesim paket sınırında olursa parçalar tek başına ayrıştırılabilir — **ölçüldü:** parçalar birleştirilince bayt bayt aynı dosya, ortadan alınan üç

parçadan 268 kare çözüldü.

- **Neden fMP4 değil:** fMP4 rastgele kesilemiyor — her parçaya `ftyp + moov` başlığının yeniden yazılması gerekirdi. TS'te böyle bir başlık yok, biçim kendi kendini senkronluyor (HLS'in TS kullanmasının da sebebi bu).
- **Süre neden duvar saatiyle ölçülüyor:** bayt sayısından süre çıkarılamıyor (bit hızı değişken); ffmpeg RTSP'yi gerçek zamanlı okuduğu için duvar saati yeterince yakın.

Video kütüphanesi için ffmpeg/ffprobe çağrıları (MediaTools):

İşlev	Komut/bayrak	Gerekçe
<code>probe()</code>	<code>ffprobe -show_format -show_streams</code>	Girdi imzalı HTTP adresi olabilir — ffprobe yalnızca gereken bölümleri range isteğiyle çeker
<code>thumbnail()</code>	<code>-ss girdiden önce</code>	ffmpeg dosyanın yalnızca gereken kısmını okur; sondan verilseydi baştan çözerek ilerler, uzun videoda dakikalar sürerdi
<code>previewClip()</code>	<code>-an , +faststart , yuv420p</code>	Sessiz, anında başlaması için faststart şart, bazı kaynaklar 10-bit/4:2:2 geliyor ve tarayıcılar oynatamıyor
<code>extractAudio()</code>	<code>-vn -ac 1 -ar 16000 -f s16le</code>	VAD/Triton'un beklediği tam biçim — canlı <code>AudioStream</code> ile aynı bayraklar
<code>remuxFastStart()</code>	<code>-c copy -movflags +faststart</code>	Yeniden kodlama yok ama dosya baştan sona okunup yazılıyor — yalnızca gerçekten gerektiğinde çağrılıyor

HLS + nginx önbellekleme

`frontend/yayin-frontend/nginx.conf` , tek origin'den (`:3000`) hem statik dosyaları hem backend API'sini hem MediaMTX'in HLS çıktısını sunuyor — CORS gerekmiyor, izleyicinin ayrıca `:8888` 'e erişmesi gerekmiyor.

`.m3u8` ve `.ts / .m4s` **AYRI location** çünkü cache ömürleri tamamen farklı: playlist saniyeler içinde değişiyor (`proxy_cache_valid 200 2s`), segment dosya adı tekil ve yazıldıktan sonra asla değişmiyor (`proxy_cache_valid 200 24h , immutable`).

proxy_cache_lock on şart: yeni segment üretilince onlarca istemci aynı ~100ms'lik pencerede gelir; bu olmadan hepsi cache-miss alıp **aynı anda** MediaMTX'e giderdi (thundering herd).

Sorgu dizesi korunmalı: MediaMTX oturum çerezi için 302 ile kendine yönlendiriyor (`?cookieCheck=1`); `proxy_pass` 'a URI parçası eklendiğinde nginx orijinal sorgu dizesini otomatik taşımadığı için `$is_args$args` açıkça eklenmesi gerekiyor — aksi halde çerez doğrulaması upstream'e hiç ulaşmaz.

Yönlendirmede \$http_host kullanılmali: `proxy_redirect` 'te `$host` yerine `$http_host` kullanılıyor; `$host` portu düşürür. MediaMTX 302 Location'ını Host başlığından kuruyor, bu yüzden frontend 80 dışında bir portta çalışırken portsuz Host doğru adrese çözülmeli.

WebSocket location'da (`/ws/`) `Upgrade / Connection` başlıkları açıkça iletiliyor — nginx varsayılan olarak hop-by-hop başlıkları düşürüyor, el sıkışma sessizce HTTP'ye düşerdi. `proxy_read_timeout 3600s` — canlı altyazıda mesajlar arası dakikalar geçebilir (sessizlik), varsayılan 60sn boşa kalma bağlantıyı koparırdı.

Triton — KServe v2 protokolü ve iç çalışma mantığı

`TritonClient.java` , Triton'un sabit KServe v2 HTTP sözleşmesiyle konuşuyor (özel bir endpoint yazılamıyor): `POST /v2/models/whisper/infer` ve `POST /v2/models/marian_en_<dil>/infer`.

Whisper (backend: "python" , max_batch_size: 8) — gerçek istek/yanıt (binary-data-extension: JSON yalnızca metadata, ham PCM byte'ları gövdenin sonuna düz ekleniyor, base64 şişmesini önlemek için):

```
POST /v2/models/whisper/infer
Content-Type: application/octet-stream
Inference-Header-Content-Length: <json başlığın byte uzunluğu>

{"inputs":[{"name":"PCM_AUDIO","shape":[1,48000],"datatype":"INT16",
  "parameters":{"binary_data_size":96000}}],
  "outputs":[{"name":"PIVOT_TEXT"}, {"name":"SOURCE_LANGUAGE"}, {"name":"LANGUAGE_CONFIDENCE"}]}
<96000 byte ham PCM verisi>
```

Yanıt:

```
{"outputs":[
  {"name":"PIVOT_TEXT","data":["I will not let you go."]},
  {"name":"SOURCE_LANGUAGE","data":["tr"]},
  {"name":"LANGUAGE_CONFIDENCE","data":[0.98]}
]}
```

`model.py` mantığı: `PCM_AUDIO` 'yu al → `int16→float32` normalize (/32768.0) → `.reshape(-1)` (`np.stack` 'in ürettiği çok boyutlu diziye `CTranslate2`'nin beklediği düz forma indirir) → öznitelik çıkar → `Tokenizer(task="translate", language="en", multilingual=True)` — **EN pivot mekanizması tam olarak burada:** `multilingual=True` sayesinde model her segmentin gerçek kaynak dilini kendi tespit edip kullanıyor, çıktıyı her zaman İngilizce'ye çeviriyor. `sequence_batching` **yok** (VAD zaten tam bölüt üretiyor, istekler arası taşınacak durum yok) ve `response_cache` **yok** (ses baytları neredeyse hiç bit-bit aynı gelmiyor, isabet ihtimali sıfıra yakın). `STT_MODEL` (whisper boyutu) yalnızca **build** zamanında okunuyor; `model.py` çalışma zamanında boyutu bilmiyor.

Marian (backend: "python" , max_batch_size: 16 , response_cache: enable: true) — gerçek istek/yanıt:

```
// İstek
{"inputs":[{"name":"SOURCE_TEXT","shape":[1,1],"datatype":"BYTES","data":["The weather is nice today."]},
  "outputs":[{"name":"TRANSLATED_TEXT"}]}

// Yanıt (model adı marian_en_<dil> – dile göre değişir)
{"model_name":"marian_en_de","outputs":[{"name":"TRANSLATED_TEXT","data":["Das Wetter ist heute schön."]}]}
```

Marian artık CTranslate2 çalıştırıyor, ONNX Runtime DEĞİL (aşağıdaki alt başlıklar önceki mimariyi de anlatıyor — geçmiş, kod hâlâ öyleymiş gibi okunmasın diye ayrı tutuldu).

Neden değişti — ölçülen VRAM büyümesi: ONNX Runtime'ın (`optimum.onnxruntime.ORTModelForSeq2SeqLM`) CUDA "caching allocator"ı, Marian'a gelen değişken cümle sayısı/uzunluğu yüzünden her yeni tensor şekli için ayrı bellek bloğu açıp bunu **hiç geri vermiyordu**. Ölçüldü: gerçek 15-kanal yükünde 1 saatte 590 MB'tan 5,1 GB'a çıkıp kartı (6,1 GB) tıkadı, çeviri başarı oranı %5-30'a düştü. Whisper (`faster-whisper` /CTranslate2) aynı sorunu hiç yaşamıyordu çünkü her girdiyi sabit 30 saniyelik pencereye dolduruyor — tensor şekli hiç değişmiyor, tek blok sürekli yeniden kullanılıyor (ölçüldü: 164 MB, saatlerce sabit). Marian'ı da CTranslate2'ye taşıyınca aynı sabit-havuz davranışını kazandı: ölçüldü, 452 MB, 3 dakikalık gerçek yükte **hiç büyümedi**, başarı oranı %100'e çıktı. **Aynı model ağırlıkları kullanılıyor** (`MARIAN_MODELS` 'taki Hugging Face repo'su değişmedi) — değişen yalnızca GPU'da nasıl çalıştığı.

Şu anki model.py mantığı (`ctranslate2.Translator`): cümle sınırlarına (`(?<=[!?])\s+`) böl → 900 karakteri aşan parçaları kelime sınırına saygılı kes (Marian cümle-seviyesinde eğitildi, uzun paragrafı sessizce kırıyor) → **tüm batch'teki tüm cümleleri tek listede düzleştirip** `MarianTokenizer.tokenize()` ile alt-kelime token'larına çevir → `translator.translate_batch(..., beam_size=1)` → `convert_tokens_to_string` ile geri çevir → cümleleri orijinal isteklerine göre yeniden grupla. `response_cache` açık (Whisper'in aksine kapalı) — haber içeriğinde dolgu cümleler kelimesi kelimesine tekrar ediyor, çeviri önbelleğinin gerçek bir isabet şansı var.

İki uygulama ayrırtsı (CTranslate2 dışı aktarımı için gerekli):

1. **ctranslate2 4.8.1 + transformers 4.48.0 sürüm uyumsuzluğu** — `TransformersConverter` her zaman `from_pretrained(..., dtype=...)` çağırıyor ama bu `transformers` sürümü `MarianMTModel` 'de böyle bir parametre tanıtmıyor (`TypeError`). Çözüm: `export_models.py` 'de küçük bir alt sınıf (`_DtypeDuzeltmisConverter`) bu kwarg'ı çağırmadan önce siliyor.

2. CTranslate2'nin Marian config'i `add_source_eos: false` — kaynak metnin sonuna EOS token'ini CTranslate2 eklemiyor, çağırılan taraf eklemek zorunda; aksi halde encoder çıkışı bozulup model sonsuz tekrara girer. `marian_model.py`'de `tokenizer.tokenize(cumle) + [tokenizer.eos_token]` satırı bunu sağlıyor.

Sağlık ucu `GET /v2/health/ready` — `strict_readiness=1` olduğu için yapılandırılmış modellerin **hepsi** yüklü değilse `400` döner; `TritonClient.saglikliMi()` bunu admin panelin "Sistem Sağlığı" kartı için kullanıyor.

WebSocket protokolü — gerçek mesaj örneği

`/ws/altyazi/{channelId}` — kimlik doğrulaması **yok**, bilinen belgelenmiş bir eksik (kanal ID'sini bilen herkes bağlanabilir). Her mesaj bir `SubtitleEvent`:

```
{
  "channelId": "f7209843-c9d9-47db-89d2-b299013bcbbba",
  "baslangic": "2026-08-16T19:44:00.453Z",
  "bitis": "2026-08-16T19:44:04.200Z",
  "kaynakDil": "tr",
  "metinler": {"de": "...", "ru": "..."},
  "kesik": false
}
```

`metinler` haritasındaki anahtarlar `STT_TARGET_LANGS`'a göre değişir (sabit değil). Dağıtım (`SubtitleBroadcaster`): `Map<UUID, Set<Session>>`, kanal başına bağlı session kümesi. Redis'e **tek** `psubscribe altyazi:*` aboneliği açılır ve **asla bırakılmaz** — süreç boyunca açık kalan tek bir abonelik, izleyici gidiş-gelişinde bırakıp yeniden açmanın getireceği gecikmeyi ortadan kaldırıyor (bkz. S2 "Container'lar" → `redis`).

Frontend Teknoloji Yığını

React 19.2.7 + TypeScript ~6.0.2 + Vite 8.1.1. `package.json`'da neden bu sürümlerin seçildiğine dair bir yorum yok — gözlemsel: hepsi yazım anındaki en güncel kararlı sürümler.

UI katmanı — shadcn/ui kalıbı, kütüphane değil: `@radix-ui/react-dialog`, `-label`, `-select`, `-slot` (erişilebilir, stilsiz davranış primitifleri)

- `class-variance-authority` + `clsx` + `tailwind-merge` — bu dördü birlikte "shadcn/ui" olarak bilinen deseni oluşturuyor: bileşenler `npm install`'la gelen bir paket değil, projenin kendi `src/components/ui/` altına kopyalanmış kaynak kod. **Neden:** tasarım sistemini tamamen kod tabanının kontrolünde tutuyor, bir UI kütüphanesinin sürüm/breaking-change döngüsüne bağımlı kalmıyor.

Diğer bağımlılıklar: Tailwind CSS 4 (`@tailwindcss/vite` — doğrudan Vite plugin'i, PostCSS ara katmanı yok), `hls.js` 1.6.16 (tarayıcıların çoğu HLS'i native desteklemediği için — yalnızca Safari destekler — MSE üzerinden yazılımsal HLS demux/oyunatma), `sonner` (toast bildirimleri), `lucide-react` (ikon seti), `babel-plugin-react-compiler` (otomatik memoization — `useMemo` / `useCallback` elle yazmak yerine derleyici build zamanında ekliyor).

Test kütüphanesi YOK. `package.json`'da `test` script'i yok, hiçbir `*.test.tsx` dosyası yok. Doğrulama tamamen manuel: `docker compose build frontend` (gerçek `tsc -b && vite build`) ile derleme hatası kontrolü + tarayıcıda gerçek akış testi.

State yönetimi — kütüphanesiz, Context API + useState : Redux/Zustand/Recoil/Jotai gibi bir state kütüphanesi kullanılmıyor. İki desen var: (1) **global oturum durumu** (`AuthContext.tsx`) — düz React `createContext` + `useState`; token yenileme başarısız olduğunda `setSessionEndedListener` callback'ile oturum tek yerden düşürülüyor. (2) **Sayfa-yerel veri** — her sayfa kendi `useState` + `useEffect(() => { fetch... }, [])` çiftini yazıyor, global bir "sunucu durumu" katmanı (React Query/SWR gibi) yok. Otomatik tazeleme gereken sayfalarda elle `setInterval` kuruluyor (ör. `AdminSistemLoglarPage`, `AdminGenelBakisPage` — 15 saniyede bir). **Neden bu kadar yalın:** kod tabanında açık bir gerekçe yorumu yok; gözlemsel çıkarım — sayfa sayısı ve paylaşılan sunucu durumu az, bir state kütüphanesinin karmaşıklığı gerekçelendirilmemiş.

Routing: `react-router-dom` 7, tek bir `<Routes>` ağacı — `/giris` ve `/yetkisiz` herkese açık, geri kalan her şey `<RequireAuth />` (oturum şartı) altında. Admin rotaları (`/yonetim/*`) ikinci bir `<RequireAuth roles={['Yonetici']} />` katmanı ile ayrıca sarılı — rol kontrolü route seviyesinde, sayfa içinde değil. İki ayrı kabuk: `AppLayout` (izleme uygulaması) ve `AdminLayout` (yönetim paneli), birbirinden tamamen bağımsız.

Docker imajı — iki aşamalı build:

```
FROM node:22-alpine AS build # derleme araçları burada kalır
RUN npm ci && npm run build
FROM nginx:1.27-alpine # yalnızca dist/ çıktısı buraya geçer
```

Neden iki aşama: `node_modules` (~300 MB) son imaja hiç girmiyor. `package.json` / `package-lock.json` kaynak koddan ÖNCE kopyalanıyor ki `npm ci` katmanı Docker'ın layer cache'inden gelsin. **Neden `nginx`, `Vite`'in kendi `preview` sunucusu değil:** `nginx.conf` yalnızca statik dosya sunmuyor — backend'e (`/api/`, `/ws/`, `/docs`) ve MediaMTX'e (`/hls/`) **reverse proxy** yapıyor; tarayıcı tek origin'den (`:3000`) konuştuğu için CORS hiç devreye girmiyor.

Rehberli tur sistemi — bağımlılıksız, elle yazılmış: `components/tour/` altında `react-joyride` gibi bir kütüphane **kullanılmıyor** — spotlight overlay (`box-shadow: 0 0 9999px`), adım yönetimi, `localStorage` tabanlı "bir kez göster" mantığı elle yazılmış. Kod içinde bunun için yazılı bir gerekçe **yok** — gözlemsel çıkarım: host makinede Node/npm kurulu olmadığı için yeni bir npm bağımlılığı eklemek `package-lock.json` 'ı host'ta güncelleyemeyeceğinden Docker build'ini kırılmaya açık bırakırdı.

Canlı oynatıcı — geri sarma, ekran görüntüsü, klip mimarisi

`PersistentPlayers.tsx` 'teki her karo, canlı ile geri sarılmış (rewound) durum arasında **iki farklı `<video>` elementi** arasında geçiş yapıyor:

```
rewindUrl === null → <HlsPlayer> (hls.js, canlı akış, kendi <video>'su)
rewindUrl !== null → düz <video src={rewindUrl}> (LiveRewind'in indirdiği mp4 blob)
```

`rewindUrl`, kullanıcı denetim çubuğundaki "-10 sn" ile canlı HLS tamponunun (~14 sn) dışına çıktığında `LiveRewind` bileşeninin DVR'dan çektiği bir bölümün `blob: URL`'i oluyor (`handleBufferExceeded` → `liveRewindRef.current.seekTo(...)`). Geri sarılmış parçanın da başına gelirse `rewindStart` (o parçanın başladığı mutlak an) sayesinde bir önceki parçaya **zincirlenerek** devam edilebiliyor.

`TileActions` 'ın kare yakalama/kayıt tutamağı (`captureRef`) canlı ile rewind arasında **doğru video elementine yönlendirilmek zorunda** — `HlsPlayer` rewind'e geçilince tamamen unmount olduğu için tutamağın canlı/rewind durumuna göre doğru hedefi seçmesi gerekiyor:

- Ekran görüntüsü, canlı/rewind durumuna göre doğru video elementini ve doğru `playingDate()` hesaplamasını** (`rewindStart + video.currentTime`) seçen bir `tileCapture` nesnesi üzerinden alınıyor (`PersistentPlayers.tsx`); `HlsPlayer` 'ın kendi tutamağı yerine bu nesne `TileActions` 'a veriliyor.
- DVR sayfasında (`DvrPage.tsx`) da aynı ekran görüntüsü özelliği var** — backend zaten kanaldan bağımsız çalışıyor: `ScreenshotService.capture()` 'daki yorum birebir "*İstemci geleceğe ait bir an bildirmesin; geçmiş serbest (geriye sarmadan yakalanabiliyor).*" diyor. Aynı video → canvas → blob tekniğiyle (`grabFrame` 'in filmstrip için kullandığıyla aynı) bir düğme sunuluyor; `playFrom()` segmenti aynı-kökenli bir `blob: URL` olarak indirdiği için canvas "tainted" olmuyor.
- Rewind'de "kayda başla", canlıyı değil izlenen geçmiş anı kaydediyor.** `recordingsApi.start/stop` her zaman "şimdi"den başladığı ve geçmiş bir andan başlamayı desteklemediği için `recordingsApi` 'ye hiç dokunulmuyor: `TileActions` içinde `pendingClipStart` adında yalnızca istemcide tutulan bir state var — "başla" rewind'deyken `capture.current.playingDate()` 'i (görünen anı) hatırlıyor, "durdur"a basıldığında bu andan **yine `capture.current.playingDate()` 'e** (durdurma anında görünen an — `new Date()` DEĞİL) kadar `clipsApi.create(...)` ile DVR'dan doğrudan klip isteniyor — başlangıç ve bitiş simetrik, ikisi de "o an ekranda ne gösteriliyorsa" o, kullanıcı hâlâ rewind'deyken durdursa bile.

Üçü de DVR'ın **zaten sürekli kayıt tuttuğu** gerçeğine dayanıyor — `pendingClipStart` akışı canlı-yayın-odaklı `recordingsApi` yerine DVR-odaklı `clipsApi.create` kullanarak geçmişten başlamayı ekstra bir backend değişikliği gerektirmeden çözüyor.

3. Kurulum ve Başlatma

Gereksinimler

- Docker + Docker Compose

- NVIDIA GPU + `nvidia-container-toolkit` (Triton'ın GPU modunda çalışması için — CPU'da da çalışır ama canlı altyazı üretim gecikmesi CPU'da 22-32 sn'ye çıkıyor, izleyicinin HLS gecikmesini (~6-12 sn) aştığı için altyazı hiç görünmez)
- Bu depoda **Maven/Node.js host'ta kurulu olmak zorunda değil** — build Docker multi-stage imajların içinde yapıyor (`./mvnw` / `npm ci` container içinde çalışıyor)

`gereksinimler.sh` eksik olanı raporluyor (`--kur` ile kuruyor).

Lokal çalıştırma adımları

```
git clone <repo>
cd Yayin_Platformu

./gereksinimler.sh # eksik bağımlılığı söyler, hiçbir şey kurmaz (--kur ile kurar)
./yapilandir.sh    # donanımı (GPU/VAAPI/yazılım) otomatik tespit edip .env üretir
./baslat.sh        # imajları kurar (Triton dahil, ~51 GB, ilk seferde yavaş) ve başlatır
```

`.env` zaten varsa `yapilandir.sh` üzerine yazmaz — yanlış tespit durumunda `.env` 'i elle düzeltip `./baslat.sh` 'i tekrar çalıştırmak yeterli.

```
./yapilandir.sh --zorla # .env'i sıfırdan yeniden üret
./baslat.sh --yeniden  # imajları sıfırdan kurarak başlat
./baslat.sh --durdur   # durdur (veri korunur)
./baslat.sh --sifirla  # durdur ve TÜM VERİYİ sil – GERİ ALINAMAZ
```

Compose dosyası kökte, `-f` gerekmiyor: `docker compose up -d` / `down` / `logs -f <servis>` doğrudan çalışır. Elle, script'siz kurulum adımları için `README.md` → "Elle kurulum" bölümüne bakın.

Açılıştan sonra erişim adresleri:

	Adres
Arayüz	<code>http://localhost:3000</code>
API belgesi (Swagger UI)	<code>http://localhost:8090/docs</code>
Keycloak	<code>http://localhost:8080</code> — <code>admin</code> / <code>admin</code>
MinIO konsolu	<code>http://localhost:9001</code>
Grafana	<code>http://localhost:3001</code> — <code>admin</code> / <code>admin</code>

Uygulama kullanıcılarının ilk şifresi **12345678** (`realm-export.json` 'a gömülü, admin panelden değiştirilebilir).

Yapılandırma (öne çıkan `.env` alanları)

Tüm alanların tam listesi `README.md` → "`.env` alanları" bölümünde. Öne çıkanlar:

Değişken	Ne işe yarar
<code>CHANNELS_ENCODER</code> / <code>VIDEOS_ENCODER</code>	<code>NVENC</code> <code>VAAPI</code> <code>YAZILIM</code> — donanım kodlayıcı seçimi
<code>STT_TARGET_LANGS</code>	Canlı altyazı hedef dilleri, virgülle ayrılmış, sınırsız sayıda (<code>tr,de,ru,fr,...</code>)
<code>MARIAN_MODELS</code>	Zorunlu — her hedef dil için <code>kod=Hugging Face repo</code> eşlemesi, eksik dil build'i durdurur (varsayılan kalıp tahmin edilmiyor, bkz. <code>triton/export_models.py</code>)
<code>MARIAN_INSTANCES</code> / <code>WHISPER_INSTANCES</code>	Model başına paralel GPU kopyası — rebuild gerekmez , <code>docker compose up -d triton</code> yeterli. Dikkatli artırın : ölçüldü — tek dilde <code>MARIAN_INSTANCES=de=2</code> VRAM'i 5GB'a çıkarıp (2918+2132 MiB, iki kopya) kartı (6141 MiB) doldurdu, 15 kanal yükünde <code>cudaErrorInvalidDevice</code> /OOM ile çöktü. <code>de=1</code> 'e dönünce VRAM 1,2GB'a düştü

Değişken	Ne işe yarar
ALTYAZI_HLS_GERIDE	İzleyiciyi canlı kenardan geriye alır = altyazının bütçesi ($\text{bütçe} = \text{ALTYAZI_HLS_GERIDE} \times \text{bölüt süresi}$, kural: bütçe $\geq$ p95 gecikme, bkz. §2 "Veri akışı")
STORAGE_*_RETENTION	Klip/ekran görüntüsü/altyazı saklama süresi (P30D , 720h , 0 =kapalı)

Backend, `.env`'i **kendisi okur** ve tarayıcının ihtiyaç duyduğu bazı değerleri (`ALTYAZI_HLS_GERIDE` , `STT_TARGET_LANGS`) kimlik istemeyen `GET /api/ayarlar/oynatici` ucundan sunar — bu sayede bu tür bir ayarı değiştirmek `docker compose build frontend` gerektirmez, yalnızca backend yeniden başlatılır.

4. Veri Modeli ve Veritabanı

Varlıklar (özet)

Varlık	Modül	Not
<code>AppUser</code> , <code>Role</code>	<code>user</code>	Keycloak realm rolleriyle senkron (bkz. <code>Roles.java</code>)
<code>Channel</code>	<code>channel</code>	Kanal tanımı, MediaMTX path eşlemesi
<code>Radio</code>	<code>radio</code>	Kanalla simetrik, ses-yalnız yayın
<code>Clip</code> , <code>ScheduledRecording</code> , <code>ActiveRecording</code>	<code>clip</code>	Klip çıkarma, planlı kayıt, o anki manuel kayıt
<code>DvrSegment</code>	<code>dvr</code>	7 günlük geriye sarma segmentleri
<code>Video</code>	<code>video</code>	Yüklenen video, video altyazısı
<code>Subtitle</code>	<code>subtitle</code>	Altyazı satırı — dil bazlı metinler <code>Map<String,String></code> (JSON) sütununda, sabit tr/de/ru sütunu YOK , bu yüzden dil sayısı şema değişikliği gerektirmeden büyüyebiliyor
<code>Screenshot</code>	<code>screenshot</code>	Ekran görüntüsü galerisi
<code>EtkinlikKaydi</code>	<code>etkinlik</code>	Admin panel "etkinlikler" denetim izi

Tam paket/dizin haritası: `src/main/java/org/example/*/entity/`.

İlişkiler (ER)

Tüm ilişkiler tek yönlü `@ManyToOne` (child → parent) — hiçbir yerde `@OneToMany` / `@ElementCollection` koleksiyonu yok, Panache'in active-record felsefesiyle tutarlı (parent kendi child'larını taşımıyor, sorgu child tarafından açılıyor):

<code>AppUser</code>	→ <code>Role</code>
<code>Channel</code>	→ <code>AppUser</code> (created_by)
<code>Radio</code>	→ <code>AppUser</code> (created_by)
<code>Video</code>	→ <code>AppUser</code> (uploaded_by)
<code>Clip</code>	→ <code>Channel</code> (nullable, kanal silinse de klip kalır) + <code>AppUser</code> (requested_by)
<code>Screenshot</code>	→ <code>Channel</code> (nullable) + <code>AppUser</code> (captured_by)
<code>Subtitle</code>	→ <code>Channel</code> (NOT NULL)
<code>DvrSegment</code>	→ <code>Channel</code> (NOT NULL)
<code>ActiveRecording</code>	→ <code>Channel</code> + <code>AppUser</code> (ikisi de NOT NULL)
<code>ScheduledRecording</code>	→ <code>Channel</code> + <code>AppUser</code> (NOT NULL) + <code>Clip</code> (nullable, henüz kırılmadıysa)
<code>EtkinlikKaydi</code>	→ FK YOK; kullanıcı_id/kullanıcı_adi düz kolon, hedef_turu+hedef_id ile herhangi bir tabloya polimorfik referans (denetim izinin kalıcı kalması için – hedef silinse bile kayıt bozulmaz)

Görsel şema — aynı ilişkiler, iki merkez varlık (`AppUser` , `Channel`) etrafında:

Role



| (rol ataması)

AppUser



|— created_by ————— Channel, Radio
|— uploaded_by ————— Video
|— captured_by ————— Screenshot (channel_id nullable)
|— requested_by ————— Clip (channel_id nullable)
|— (NOT NULL) ————— ActiveRecording, ScheduledRecording

Channel



|— channel_id (NOT NULL) — Subtitle, DvrSegment,
| ActiveRecording, ScheduledRecording
|— channel_id (nullable) — Clip, Screenshot

ScheduledRecording — clip_id (nullable, henüz kırılmadıysa) → Clip

EtkinlikKaydi — FK YOK — hedef_turu + hedef_id ile yukarıdaki tabloların
herhangi birine polimorfik, serbest referans
(hedef silinse bile denetim kaydı bozulmaz)

Clip.channel_id ve Screenshot.channel_id 'nin **nullable** olması bilinçli bir tasarım kararı (V21 migration): kanal silinince ürettiği klip/ekran görüntüleri kaybolmuyor, yalnızca kanal referansı boşalıyor.

Subtitle dil bazlı metinleri sabit tr / de / ru sütunu yerine Map<String,String> (JSON) sütununda tutuyor — bu yüzden altyazı dil sayısı büyürken şema değişikliği gerekmiyor.

Veritabanı geçişleri (Flyway)

src/main/resources/db/migration/ altında **V1'den V28'e** kadar sıralı SQL dosyaları (V1__users_and_roles.sql ... V28__video_altyazi.sql). Quarkus açılışta otomatik migrate ediyor (quarkus.flyway.migrate-at-start) — **yalnızca backend container'ında**; video-worker 'da QUARKUS_FLYWAY_MIGRATE_AT_START=false (aynı jar iki container'da migration yarış durumu yaratmasın diye, bkz. docker-compose.yaml).

Yeni migration eklerken: bir sonraki V<n>__ numarasını kullanın, mevcut bir migration dosyasını **asla değiştirmeyin** (checksum kırılır).

Veritabanı adı **yayin_merkezi** (yayin değil), kullanıcı **app_user** .

MinIO bucket'ları

application.properties 'te sabit: videos.bucket=videolar , clips.bucket=klipler , screenshots.bucket=ekran-goruntuleri , dvr.bucket=dvr .

5. API ve Entegrasyon Dokümantasyonu

Endpoint detayları

Tam ve güncel liste **Swagger UI**'da: http://localhost:8090/docs (OpenAPI JSON: /docs/openapi.json , quarkus-smallrye-openapi ile üretiliyor).

REST kaynak sınıfları (src/main/java/org/example/**/*Resource.java):

Sınıf	Yol	Erişim
AuthResource	/api/auth	@PermitAll (giriş)
CurrentUserResource	/api/users/me	Herkes (oturum açmış)
AdminUserResource	/api/admin/users	Roles.YONETICI
ChannelResource	/api/channels	Karma (izleme herkese, yönetim role bağlı)
RadioResource	/api/radios	Kanalla simetrik
ClipResource, ChannelClipResource	/api/clips, /api/channels/{id}/clips	—
ScheduledRecordingResource, ChannelScheduledRecordingResource	Planlı kayıt	—
DvrResource	/api/channels/{channelId}/dvr	—
VideoResource	/api/videos	—
ScreenshotResource	Ekran görüntüsü galerisi	—
ChannelSubtitleResource	Altyazı geçmişi/WebSocket tetikleyici	—
OynaticiAyarResource	/api/ayarlar/oynatici	@PermitAll — kimlik istemez
AdminAnalitikResource, AdminEtkinlikResource, AdminSistemLogResource	Admin panel	Roles.YONETICI

Triton/WebSocket istek-yanıt örnekleri ve iç çalışma mantığı: §2.

Paket paket istek/yanıt referansı

Aşağıda `src/main/java/org/example/` altındaki **her paket**, gerçek istek/yanıt örnekleriyle. Zaten §2'de (Triton protokolü, mimari kalıplar) veya §4'te (veri modeli) anlatılmış konular burada tekrarlanmıyor, yalnızca işaret ediliyor.

auth

Dosyalar: `AuthResource` (uç noktalar), `AuthService` (Keycloak'a proxy mantığı), `KeycloakTokenClient` (Keycloak OIDC token uçlarına REST Client arayüzü), `dto/{LoginRequest,RefreshRequest,LogoutRequest,TokenResponse}`.

Uygulama kimlik bilgisi tutmaz — üçü de `@PermitAll`, gerçek doğrulama Keycloak'ta yapılır. `AuthService`, `KeycloakTokenClient` üzerinden Keycloak'un `/realms/{realm}/protocol/openid-connect/token` ve `/logout` uçlarına `client_id` + `client_secret` ekleyerek form-urlencoded istek atar; Keycloak'un ham OAuth2 yanıtı `TokenResponse` 'a birebir eşlenip aynı alan adlarıyla dışarı verilir.

Metot + Yol	Yetki	İstek	Yanıt
POST /api/auth/login	@PermitAll	LoginRequest	TokenResponse
POST /api/auth/refresh	@PermitAll	RefreshRequest	TokenResponse
POST /api/auth/logout	@PermitAll	LogoutRequest	204

```
// POST /api/auth/login
{"username": "moderator1", "password": "gizliSifre123"}
```

```
// 200 OK
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_in": 300,
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_expires_in": 1800,
  "token_type": "Bearer"
}
```

Hatalar `AppException` üzerinden aynı `ErrorResponse` şekline dönüşür: geçersiz kimlik bilgisi `401`, Keycloak'un kendisi ulaşılamazsa `502`. `login()` her denemeyi (başarılı/başarısız) `EtkinlikService` üzerinden denetim izine yazar.

`user`

Dosyalar: `CurrentUserResource` (`/api/users/me`), `AdminUserResource` (`/api/admin/users`), `UserService` (Keycloak Admin Client + yerel `AppUser / Role`), `UserProvisioningService / Filter` (ilk girişte yerel satır oluşturma), `Roles`, `TokenClaims`, `entity/{AppUser,Role}`.

`UserService`, kullanıcı CRUD'unu Keycloak'ta yapar; yerel `users` tablosu yalnızca `keycloak_id` (= `sub`) ile eşlenen bir ayna. `UserDto.id` her zaman Keycloak kullanıcı id'sidir — yerel `users.id` hiçbir API'de görünmez.

Metot + Yol	Yetki	İstek	Yanıt
GET <code>/api/users/me</code>	Authenticated	—	<code>UserDto</code>
PUT <code>/api/users/me/password</code>	Authenticated	<code>ChangePasswordRequest</code>	<code>204</code>
GET <code>/api/users/me/kota</code>	Authenticated	—	<code>QuotaService.Usage</code>
GET <code>/api/admin/users?search=&first=0&max=20</code>	YONETICI	—	<code>UserDto[]</code>
POST <code>/api/admin/users</code>	YONETICI	<code>CreateUserRequest</code>	<code>201</code> + <code>UserDto</code>
PUT <code>/api/admin/users/{id}/role</code>	YONETICI	<code>UpdateRoleRequest</code>	<code>UserDto</code>
PUT <code>/api/admin/users/{id}/password</code>	YONETICI	<code>ResetPasswordRequest</code>	<code>204</code>
DELETE <code>/api/admin/users/{id}</code>	YONETICI	—	<code>204</code>
POST <code>/api/admin/users/sync</code>	YONETICI	—	<code>SyncResultDto</code>

```
// GET /api/users/me → 200
{
  "id": "3fa1c2e0-1111-4a2b-9c3d-abcdef123456",
  "username": "moderator1", "email": "mod1@ornek.com",
  "firstName": "Ayşe", "lastName": "Yılmaz",
  "enabled": true, "role": "Moderatör",
  "createdAt": "2026-06-01T09:12:00Z"
}
```

```
// POST /api/admin/users/sync → 200
{
  "created": ["yeniKullanici7"],
  "updated": ["ayseY"],
  "orphaned": ["silinmisKullanici3"]
}
```

`{id}` her zaman Keycloak `sub`. `sync()` Keycloak'taki tüm kullanıcıları yerel `users` tablosuyla karşılaştırır; `orphaned` (Keycloak'ta artık olmayan yerel kayıtlar) **otomatik silinmez**, yalnızca raporlanır.

channel

Dosyalar: `ChannelResource`, `ChannelService`, `ChannelDeletionService`, `ChannelRestorer`, `MediaMtxClient` / `MediaMtxService`, `HlsPlaylist` / `Rendition` / `SourceProbe` / `TranscodeCommand`, `entity/Channel`.

Sınıf düzeyi `@Authenticated`; yazma uçları ayrıca `@RolesAllowed({YONETICI, MODERATOR})`.

Metot + Yol	Yetki	İstek	Yanıt
GET /api/channels	okuma	—	ChannelDto[]
GET /api/channels/{id}	okuma	—	ChannelDto
POST /api/channels	YONETICI MODERATOR	CreateChannelRequest	201 + ChannelDto
PUT /api/channels/{id}	YONETICI MODERATOR	UpdateChannelRequest	ChannelDto
POST /api/channels/{id}/kalite	okuma	{kalite}	204 (yalnızca telemetri)
GET /api/channels/{id}/silme-ozeti	YONETICI MODERATOR	—	ChannelDeletionSummary
POST /api/channels/{id}/silme	YONETICI MODERATOR	DeleteChannelRequest	204
POST /api/channels/restore	YONETICI MODERATOR	—	{restored}

```
// POST /api/channels
{
  "name": "TRT Haber",
  "sourceUrl": "rtsp://kaynak.ornek.com:554/trthaber",
  "mediamtxPath": "trt-haber",
  "active": true, "dvrEnabled": true,
  "renditions": "720p|1280x720|2500k,480p|854x480|1000k"
}
```

Kalite (rendition) değişimi ayrı bir sunucu çağrısı **gerektirmez** — `hls.js`, `ChannelDto.hlsUrl` 'deki master playlist'ten kendisi bir varyant seçer. `POST /{id}/kalite` yalnızca **telemetri** amaçlıdır — hangi kaliteyi seçtiğini denetim izine yazar, oynatıcının kendi akışını etkilemez.

Silme iki adımlı: önce dökümü göster, sonra şifreyle onaylat.

```
// GET /api/channels/{id}/silme-ozeti → 200
{
  "channelName": "TRT Haber", "clipCount": 12, "screenshotCount": 34,
  "dvrSegmentCount": 20160, "dvrHours": 168.0,
  "dvrBytes": 487213834240, "clipBytes": 1520000000, "streaming": true
}
```

```
// POST /api/channels/{id}/silme
{"password": "kendiSifrem", "deleteClips": false, "deleteScreenshots": true, "deleteDvr": false}
```

`password` her zaman **işlemi yapanın kendi** şifresidir (Keycloak'a karşı doğrulanır);

`deleteClips` / `deleteScreenshots` / `deleteDvr` bağımsız seçilir — `false` olan içerik kanaldan **kopar** ama silinmez (`Clip.channel_id` / `Screenshot.channel_id` nullable, bkz. §4).

Kapasite sınırı: `channels.max-active` (env `CHANNELS_MAX_ACTIVE`, varsayılan **16**) — aynı anda **aktif/yayında** olabilecek kanal üst sınırı. `ChannelService.requireCapacity()` bunu açık bir `409` ya çeviriyor; MediaMTX kendisi bu sınırı uygulamıyor, aşıldığında sessizce kabul edip **tüm** kanallarda bozulmaya yol açar. Bu sınır `vad.max-channels` 'tan (§5 "VAD" paketi) **bağımsız** — biri yükseltilip diğeri yükseltilmezse fazla kanallar yayınlanır ama altyazı almaz (ölçüldü, bkz. §8 "Tur 3").

radio

Dosyalar: `RadioResource`, `RadioService`, `RadioRestorer`, `AudioBridgeCommand`, `RadioSourceKind`, `entity/Radio`. Yapı `channel` ile simetrik — aynı yetki deseni, aynı `MediaMtxService` / `ViewerPresence` bağımlılıkları.

`sourceKind` istemciden **açıkça** alınır, adresten tahmin edilmez: `DOGRUDAN` (HLS/RTSP/RTMP/SRT/UDP/WHEP, adres doğrudan MediaMTX'e yazılır) veya `KOPRU` (Icecast/Shoutcast MP3 gibi MediaMTX'in okuyamadığı kaynaklar; ffmpeg araya girip AAC'ye kodlayıp RTSP üzerinden basar, ölçülen maliyet radyo başına %2,6 CPU).

```
// POST /api/radios (KOPRU – Icecast MP3)
{
  "name": "Radyo X", "sourceUrl": "http://kaynak.ornek.com:8000/radyox.mp3",
  "sourceKind": "KOPRU", "mediamtxPath": "radyo-x",
  "bitrate": "128k", "active": true, "sortOrder": 5
}
```

`bitrate` yalnızca `KOPRU` modunda anlamlıdır (ffmpeg'in AAC kodlama hedefi). `listeners` / `viewers` alanları MediaMTX'in reader sayısı DEĞİL — `ViewerPresence` tabanlı (bkz. aşağıda `viewer` paketi).

clip — kayıttan klip çıkarma

Dosyalar: `ChannelClipResource` (klip oluşturma + manuel kayıt, `/api/channels/{channelId}/clips`), `ClipResource` (`/api/clips`), `ScheduledRecordingResource` + `ChannelScheduledRecordingResource` (planlı kayıt), `ClipService`, `RecordingService`, `ScheduledRecordingService` + `Scheduler`, `ClipQueue` (Redis kuyruğu), `ClipConsumer` (tüketici havuzu), `ClipWorker` (asıl işi yapan birim), `ClipStorage`, `ChannelRecordingGate` (kanal başına eşzamanlı manuel kayıt kısıtı), `entity/{Clip,ActiveRecording,ScheduledRecording}`.

Neden `ChannelClipResource` ayrı bir sınıf: JAX-RS isteği en uzun eşleşen kaynak sınıfına yönlendirir;

`/api/channels/{id}/clips` bir `/api` kaynağında dursaydı `ChannelResource` 'a düşer, orada karşılığı olmadığı için 500 dönerdi.

Metot + Yol	Yetki	İstek	Yanıt
POST <code>/api/channels/{channelId}/clips</code>	Authenticated	<code>CreateClipRequest</code>	202 + <code>ClipDto</code>
POST <code>/api/channels/{channelId}/clips/kayit</code>	Authenticated	—	200 + <code>ActiveRecordingDto</code>
DELETE <code>/api/channels/{channelId}/clips/kayit</code>	Authenticated	—	202
GET <code>/api/clips?channelId=&origin=</code>	Authenticated	—	<code>ClipDto[]</code>
GET <code>/api/clips/{id}/links</code>	Authenticated	—	<code>ClipService.ClipLinks</code>
DELETE <code>/api/clips/{id}</code>	Authenticated	—	204
POST <code>/api/channels/{channelId}/planli-kayitlar</code>	Authenticated	<code>CreateScheduledRecordingRequest</code>	202

```
// POST /api/channels/{channelId}/clips (zamanlar UTC, clips.max-duration ile sınırlı)
{ "start": "2026-08-17T10:00:00Z", "end": "2026-08-17T10:02:30Z" }
```

```
// GET /api/clips/{id}/links → 200
{
  "stream": "https://minio.../klipler/...mp4?X-Amz-Signature=...",
  "download": "https://minio.../klipler/...mp4?response-content-disposition=attachment&...",
  "fileName": "TRT-Haber-2026-08-17-10-00-00.mp4",
  "subtitles": [{ "lang": "de", "url": "https://minio.../...-altyazi-de.vtt?..." }]
}
```


Asenkron klip üretim akışı (hem **ARALIK** hem **MANUEL_KAYIT** /planlı kayıt için ortak):

```
İstek (create) → Postgres: clips satırı (BEKLIYOR)
    → ClipQueuedEvent (transaction commit SONRASI)
    → ClipQueue.publish(): Redis LPUSH clips:bekleyen

ClipConsumer (N işçi) → BLMOVE bekleyen→isleniyor
    → worker.claim() (SKIP LOCKED)
    → worker.process(id)

DB süpürücü (clips.sweep-interval) – Redis kaçırırsa BEKLIYOR satırları
burada bulunur (güvenlik ağı).

ClipWorker.process():
1. (ARALIK dışıysa) dvrBekle(): istenen aralık DVR çizelgesine düşene
    kadar bekle, aralığı kayda kırp
2. dvrService.extractStream(...) – MediaMTX'ten AKIŞ HALİNDE çek
3. storage.put(...) – MinIO'ya akış halinde yaz
4. altYaziUret(...) – varsa DVR segmentlerinin alt yazılarından WebVTT
5. markReady(): status=HAZIR, sizeBytes, subtitleLangs
6. hata olursa: attempts++, MAX_ATTEMPTS'e kadar yeniden dene

İstemci sonucu NASIL öğreniyor: WebSocket YOK – periyodik GET /api/clips
yoklaması (Klipler sayfası).
```

Manuel kayıt akışı: **POST .../clips/kayit** bir **ActiveRecording** satırı açar; **DELETE .../clips/kayit** bitiş anını (sunucuda, "şimdi") belirleyip bir **Clip** (origin=**MANUEL_KAYIT**) oluşturur ve yukarıdaki aynı kuyruğa sokar. Planlı kayıt (**ScheduledRecordingScheduler**) başlangıç/bitiş anlarını zamanlanmış bir işle tetikleyip aynı akışa bağlanır.

dvr — geriye sarma ve zaman çizelgesi

Dosyalar: **DvrResource** (/api/channels/{channelId}/dvr), **DvrService** , **ChannelDvrRecorder** + **DvrRecorder** + **SegmentStream** (sürekli kayıt), **DvrSignal** + **DvrSignalEvent** (Redis pub/sub ile kesme sinyali), **DvrStorage** , **entity/DvrSegment** .

Okuma uçları **giriş yapmış herkese açık** — izleyici geriye sarmayla zaten aynı içeriği görebiliyor.

Metot + Yol	İstek (query)	Yanıt
GET .../dvr/timeline?from=&to=	ISO-8601 from / to	TimelineSpan[]
GET .../dvr/stream?start=&duration=	ISO-8601 start , saniye duration	video/mp4 akışı

```
// GET .../timeline?from=2026-08-17T08:00:00Z&to=2026-08-17T10:00:00Z
[
  { "start": "2026-08-17T08:00:00Z", "end": "2026-08-17T08:42:10Z" },
  { "start": "2026-08-17T08:45:00Z", "end": "2026-08-17T10:00:00Z" }
]
```

(8:42-8:45 arası listede yok = o aralıkta kayıt yok.)

stream yanıtı **movflags=frag_keyframe+empty_moov** ile **parçalı** (fragmented) MP4 — normal MP4'ün sona dönüp **moov** kutusu yazma alışkanlığı akış halinde uygulanamadığı için. Her çağrı **DVR_GERI_SARILDI** etkinliği yazar.

Sürekli kayıt (arka plan, istek/yanıt değil):

```
ChannelDvrRecorder → ffmpeg -c copy -f mpegts -  
→ SegmentStream (188-bayt paket sınırında böler)  
→ DvrStorage.put(...) her segment MinIO'ya + Postgres'te DvrSegment satırı  
→ DvrSignal (Redis pub/sub, KES): RecordingService.stop() segmenti HEMEN  
    kapatıp çizelgeye erken düşürmek için kullanır.
```

video — video kütüphanesi

Dosyalar: `VideoResource`, `VideoService`, `VideoQueue` + `VideoConsumer` + `VideoWorker` (clip ile aynı Redis kuyruk deseni),
`video/subtitle/ VideoSubtitleQueue` + `VideoSubtitleConsumer` + `VideoSubtitleWorker` (AYRI kuyruk — Triton/GPU'ya bağlı,
dakikalar sürebilir), `VideoStorage`, `MediaTools`, `entity/Video`.

Okuma `Authenticated`, yükleme/düzenleme/silme `@RolesAllowed({YONETICI, MODERATOR})`.

Yükleme akışı — uçtan uca, iki adımlı (dosya backend'den GEÇMEZ):

1. POST `/api/videos` { title, fileName, contentType?, sizeBytes }
→ kota kontrolü → Video satırı (YUKLENIYOR)
→ 201 + UploadTicket { videoId, uploadUrl, contentType, expiresAt }
2. Tarayıcı: PUT `<uploadUrl>` (doğrudan MinIO'ya)
3. POST `/api/videos/{id}/tamamlandi`
→ storage.stat(objectKey) ile GERÇEKTEN var mı doğrulanır
→ Video.status = ISLENIYOR → VideoQueuedEvent → Redis LPUSH
4. VideoWorker.process():
a. media.probe(source) b. media.thumbnail(...) c. media.previewClip(...)
d. VideoStorage.put(...) e. markReady(): HAZIR
5. (videos.subtitle-enabled=true ise, AYRI kuyruk)
VideoSubtitleWorker: ses çıkar → VAD → Triton → WebVTT → MinIO
→ Video.subtitleStatus = HAZIR

```
// POST /api/videos/{id}/izleme-ozeti (10 dilimlik ısı haritası için)  
{  
  "ziyaretEdilenDilimler": [0, 1, 2, 5, 9],  
  "tamamlandi": true, "duraklatmaSayisi": 2, "sarmaSayisi": 1, "sureMs": 184200  
}
```

screenshot — ekran görüntüsü galerisi

Dosyalar: `ScreenshotResource`, `ScreenshotService`, `ScreenshotStorage`, `entity/Screenshot`. Kare **tarayıcıda** yakalanır —
sunucu tarafı yakalama yok.

POST `/api/screenshots/{channelId}` — multipart/form-data :

Alan	Tip	Zorunlu	Açıklama
<code>dosya</code>	dosya (JPEG)	evet	Canvas'tan üretilen görsel
<code>capturedAt</code>	ISO-8601 string	hayır	İzlenen YAYIN anı — geçmiş serbest, gelecek reddedilip sunucu anına düşülür
<code>width / height</code>	integer	hayır	Piksel boyutu
<code>note</code>	string	hayır	Kullanıcı notu

```
// 201 Created
{
  "id": "9c31...", "channelId": "f720...", "channelName": "TRT Haber",
  "capturedAt": "2026-08-17T09:58:12.400Z", "width": 1280, "height": 720,
  "viewUrl": "https://minio.../ekran-goruntuleri/...jpg?X-Amz-Signature=...",
  "downloadUrl": "https://minio.../ekran-goruntuleri/...jpg?...",
  "fileName": "TRT-Haber-2026-08-17-09-58-12.jpg"
}
```

VAD — canlı altyazı boru hattı, paket paket

Sınıf	Rolü
VadConfig	Tüm sabitler tek yerde (kare boyutu, eşikler, süreler)
AudioStream	Bir MediaMTX path'inden ffmpeg ile ham PCM okuyan süreç sarmalayıcı
SileroVad	Silero VAD ONNX modelini Java içinde çalıştırıp kareye "konuşma olasılığı" skoru veren sınıf
SpeechSegmenter	Kare kare gelen skorları konuşma bölütlerine çeviren durum makinesi
SpeechSegment	Tek bir bölümü taşıyan record — STT'ye giden birim
ChannelVadWorker	Tek kanalın döngüsü: ffmpeg → kare → model → bölütleyici
TritonClient	Triton'a konuşan HTTP istemci (transcribe() , translate())
VadService	Kanal başına işçi açıp kapatan, kuyrukları yöneten, pivot/çeviri akışını orkestre eden merkezi sınıf

subtitle — REST API

GET /api/channels/{channelId}/altyazilar?from=&to= (@Authenticated) — aralıklarla **kesişen** tüm bölütleri döner:

```
[{
  "id": "b1e2...", "baslangic": "2026-08-17T09:12:40.100Z",
  "bitis": "2026-08-17T09:12:44.300Z", "kaynakDil": "tr", "guven": 0.97,
  "metinler": { "en": "...", "de": "...", "ru": "...", "es": "..." },
  "kesik": false
}]
```

POST .../altyazilar/hls-gecikme — izleyicinin tarayıcıda ölçtüğü gerçek HLS gecikmesi ({ "ms": 8400}), **altyazi.butce-ms** varsayımının yerine geçer. **POST .../altyazilar/dil** — dil seçimini denetim izine yazar ({ "dil": "de"}), akışı etkilemez.

GET /api/ayarlar/oynatici (@PermitAll):

```
{ "hlsGeride": 8, "altyaziDilleri": ["de", "ru", "es"] }
```

altyaziDilleri, **stt.target-langs** 'tan türetiliyor — backend hiç isim üretmiyor, ISO kodlarını olduğu gibi döndürüyor.

subtitle — WebSocket

ws://<host>/ws/altyazi/{channelId} (kimlik istemiyor) — her yeni **SubtitleEvent** :

```
{
  "channelId": "f7209843-c9d9-47db-89d2-b299013bcbbba",
  "baslangic": "2026-08-17T09:12:40.100Z", "bitis": "2026-08-17T09:12:44.300Z",
  "kaynakDil": "tr", "metinler": { "de": "Ich werde dich nicht gehen lassen." },
  "kesik": false
}
```

Dikkat: WebSocket'ten gelen **metinler** genelde **tek dil** içerir (o anki çağrının taşıdığı kadar) — REST'ten gelen ise o satırın **birikmiş tüm dillerini** içerir. Oynatıcı ilk açılışta REST'le geçmişti doldurur, sonrasını WebSocket'le günceller.

VideoSubtitleWorker 'ın aynı parçaları yeniden kullanımı: canlı boru hattıyla **SileroVad**, **SpeechSegmenter**, **TritonClient** **birebir paylaşılıyor**. Fark: sürekli bir **AudioStream** yerine **MediaTools.extractAudio()** ile tek seferde çıkarılmış bir PCM dosyası kare kare okunuyor, zaman damgaları duvar saati değil **Instant.EPOCH** 'tan itibaren **dosya-görelili**. Sonuç WebSocket'e değil doğrudan **WebVttWriter.yaz()** ile **.vtt** dosyasına dönüştürüp MinIO'ya yazılıyor.

etkinlik — denetim izi ve analitik

EtkinlikTuru (enum, **etkinlik_kayitlari.tur** düz metin kolonu — yeni tür eklemek migration gerektirmez, 26 değer: **GIRIS**, **IZLEME_BASLADI/BITTI**, **ALTYAZI_DIL_DEGISTI**, **KALITE_DEGISTI**, **DVR_GERI_SARILDI**, **KLIP_OLUSTURULDU**, **KANAL_EKLENDI/SILINDI**, **OYNATMA_HATASI** vb.), **EtkinlikService** (**kaydet()** / **ara()** — TEK giriş noktası, ~10 çağrı noktasında açık tek satırlık çağrılar, cross-cutting annotation yerine), **AdminEtkinlikResource**, **AdminAnalitikResource**, **AnalitikService**.

GET /api/admin/etkinlikler?tur=&kullaniciAdi=&baslangic=&first=0&max=50 (YONETICI) — sayılanmış:

```
{
  "items": [{
    "id": "b3f1...", "kullaniciAdi": "ahmet", "tur": "IZLEME_BASLADI",
    "hedefTuru": "kanal", "hedefId": "f720...", "hedefAdi": "TRT Haber",
    "detay": { "tabId": "tab-9f2...", "olusturmaZamani": "2026-08-17T14:03:11Z"
  }],
  "total": 1842, "first": 0, "max": 50
}
```

GET /api/admin/analitik/* (YONETICI) — 10 alt uç, tümü `etkinlik_kayitlari` + canlı servis sorgularından türetiliyor, ayrı bir agregasyon tablosu yok:

Yol	Yanıt
<code>/genel-bakis</code>	<code>SistemSagligiOzetDto</code> (7 bileşenlik sağlık listesi)
<code>/servis-metrikleri</code>	<code>ServisMetrikleriDto</code> (Prometheus'tan ham sayılar)
<code>/canli-durum</code>	<code>CanliDurumDto</code> (eşzamanlı izleyici/dinleyici, aktif DVR kaydı)
<code>/icerik-performansi</code>	En çok izlenen kanal/radyo/video top-10
<code>/depolama</code>	MinIO kullanım/kova dağılımı
<code>/teknik</code>	Oynatma hata/takılma oranı
<code>/genel</code>	DAU/MAU, yoğun saat dağılımı
<code>/videolar</code> , <code>/videolar/{id}</code>	Video özet listesi / tek video ısı haritası
<code>/kullanicilar/{keycloakId}</code>	Kullanıcı aktivite dialog'unun tam verisi

`viewer` — eşzamanlı izleyici sayacı

Tek dosya: `ViewerPresence`. Kanal/radyo başına, **sekme** (`tabId`) bazlı sayaç — MediaMTX'in kendi reader sayısı DEĞİL, tarayıcının periyodik heartbeat'ine dayanıyor (bir sekme kalite değişimi/hls.js'in paralel segment isteklerinde MediaMTX'e birden fazla reader gibi görünebiliyor).

- PUT /api/channels/{id}/izleyici/{tabId}** — sekme her 15 sn çağırıyor; ilk çağırıda `IZLEME_BASLADI` açılıyor.
- DELETE /api/channels/{id}/izleyici/{tabId}** — sekme kapanırken `keepalive` fetch ile; `IZLEME_BITTI` (`sureMs` ile) yazılıyor.

Bellekte, kalıcı değil — 10 sn'de bir süpürücü, 40 saniyedir heartbeat göndermeyen sekmeleri düşürüp `IZLEME_BITTI` (`sebep: sure_asi`) yazıyor.

`playback` — oynatma sağlığı telemetrisi

Tek dosya: `PlaybackHealthMetrics`. **PUT /api/{channels|radios}/{id}/oyunatma-ozeti** uçlarının arkasındaki Prometheus sayaç katmanı:

```
// istek – istemcide dakikada bir biriktirilip gönderiliyor
{"hataSayisi": 2, "takilmaSayisi": 1, "sonMesaj": "bufferStalledError"}
```

İki Prometheus sayacını (`oyunatma_hata_toplam`, `oyunatma_takilma_toplam`, `kaynak` + `ad` etiketli) artırıyor — Grafana'nın "Oynatma Sağlığı" dashboard'u burayı okuyor. `hataSayisi>0` ise ayrıca `OYNATMA_HATASI` etkinlik kaydı da açılıyor.

`sistemLog` — bir isteğin uçtan uca dönüşümü

Mimari: \$6 "İzleme ve Admin Panel". Burada tek bir isteğin tam akışı:

```
GET /api/admin/sistem-loglari?servis=triton&seviye=HATA&limit=50
  ▼ LokiClient.sorgula("{servis=\"triton\"}", 24 saat, 250)
    GET {LOKI_URL}/loki/api/v1/query_range?query=...
  ▼ Loki yanıtı → ham mesaj:
    "Failed to process the request(s) for model 'marian_en_de',
    message: RuntimeError: CUDA failed with error out of memory"
  ▼ SistemLogYorumlayici.yorumla() – KURALLAR listesinde ilk eşleşen
    regex kazanır:
    Desen: "Failed to process .* for model '(\w+)'.*out of memory"
    Şablon: "{1} modeli GPU belleği dolduğu için isteği işleyemedi."
  ▼ SistemLogDto(zaman, "triton", "HATA",
    "marian_en_de modeli GPU belleği dolduğu için isteği işleyemedi.",
    "<orijinal ham mesaj>")
  ▼ seviye=HATA filtresi + limit=50
  ▼ yanıt: SistemLogDto[]
```

Hiçbir kurala uymayan ve genel bir `error` / `fatal` / `exception` / `panic` sinyali de taşımayan satırlar `null` döner, listeye hiç girmez.

`storage`, `exception`, `media` — çapraz referans

- **storage** (`StoragePaths`, `QuotaService`, `RetentionSweeper`) — bkz. §2 "Veri Katmanı". Ek not: `QuotaService` ayrı bir sayaç tablosu tutmuyor, kullanım `clips.size_bytes` / `screenshots.size_bytes` / `videos.size_bytes` kolonlarının toplamından anlık sorgulanıyor; kota dolunca yeni iş reddediliyor, var olan veri silinmiyor. `StoragePaths` nesne anahtarlarını `<kullanıcı>/<kanal>/<id>.<uzantı>` deseninde üretiyor.
- **exception** (`AppException`, `ErrorCode`, üç `ExceptionMapper`) — bkz. §2 "Backend çatısı". `ErrorCode` 7 değer taşıyor: `NOT_FOUND` (404), `CONFLICT` (409), `UNAUTHORIZED` (401), `FORBIDDEN` (403), `UPSTREAM_ERROR` (502), `BAD_REQUEST` (400), `INTERNAL_ERROR` (500). REST uç noktası olan HER paket bu ortak sözleşmeyi kullanıyor.
- **media** (`VideoEncoder`) — bkz. §2 "Donanım kodlayıcılar". Tek dosya: `NVENC` / `VAAPI` / `YAZILIM` enum'ı, hem `channel` hem `video` paketlerinden ortak kullanılıyor.

Hata kodları ve formatı

Tüm API hataları **tek, tutarlı** bir JSON şekliyle dönüyor (`org.example.exception.ErrorResponse`):

```
{
  "timestamp": "2026-08-17T10:00:00Z",
  "status": 404,
  "error": "NOT_FOUND",
  "message": "Kanal bulunamadı: <id>",
  "path": "/api/channels/<id>",
  "fieldErrors": []
}
```

Doğrulama hatalarında `fieldErrors` doluyor (`field` + `message` çiftleri). İş kuralı hataları tek bir `AppException` sınıfından factory metotlarla fırlatılıyor (`AppException.notFound(...)`, `.conflict(...)`, `.badRequest(...)`, `.unauthorized(...)`, `.forbidden(...)`, `.upstreamError(...)`, `.internalError(...)`), her biri bir `ErrorCode` 'a (→ HTTP durum koduna) eşleniyor. Üç ayrı `ExceptionMapper` (`AppExceptionMapper`, `ValidationExceptionMapper`, `GenericExceptionMapper`) bunları aynı `ErrorResponse` şekline çeviriyor — yeni bir hata durumu eklemek için yalnızca `ErrorCode` 'a bir değer + `AppException` 'a bir factory metot eklemek yeterli, mapper'lara dokunulmuyor.

```

Resource sınıfı
|
| AppException.notFound(...) / .conflict(...) / .badRequest(...) / ...
▼
AppException (bir ErrorCode taşır)
|
| AppExceptionMapper      — AppException → ErrorResponse
| ValidationExceptionMapper — Bean Validation hatası → fieldErrors dolu
| GenericExceptionMapper  — beklenmeyen istisna → 500 ErrorResponse
|
|
▼
tek biçimli ErrorResponse JSON → istemci

```

Dış servis entegrasyonları ve kimlik doğrulama

- **Keycloak (OIDC)** — `quarkus-keycloak-authorization`. Realm: `YayinYonetimi`. API çağrıları `Authorization: Bearer <access_token>` bekliyor; Swagger UI'da `bearerAuth` şeması tanımlı (`/api/auth/login` yanıtındaki `access_token` yapıştırılıyor). **Dikkat:** host'tan alınan token `iss=localhost:8080` taşır, backend `keycloak:8080` bekler → 401. Yerel test konteyner içinden yapılmalı.

```

Host'tan login      Keycloak token'ı      backend API isteği
|                  iss=localhost:8080      |
▼                  |                        ▼
POST /api/auth/login → | → 401 (iss uyumsuzluğu)

Konteyner içinden login  Keycloak token'ı      backend API isteği
|                        iss=keycloak:8080      |
▼                        |                        ▼
POST /api/auth/login → | → 200

```

- **MinIO (S3 uyumlu)** — klip/video/ekran görüntüsü/DVR segmenti depolama, imzalı URL'lerle indirme.
- **Triton Inference Server** — KServe v2 HTTP protokolü, Java'dan `TritonClient` üzerinden. Protokol örnekleri: §2.
- **MediaMTX** — REST API (`:9997`) ile path/kayıt yönetimi, RTSP (`:8554`) ile ffmpeg akışı.

6. Dağıtım ve DevOps

CI/CD

CI/CD pipeline yok. `.github/workflows/` dizini bulunmuyor — build, test ve dağıtım tamamen elle, yukarıdaki `yapilandir.sh` / `baslat.sh` script'leri ve doğrudan `docker compose build / up` komutlarıyla yapılıyor. Bu, bu doküman yazılırken tespit edilen gerçek durum — otomatik bir pipeline eklenirse bu bölüm güncellenmeli.

Konteynerizasyon ve sunucu yapısı

7 farklı **Dockerfile**, tek amaçlı:

Dockerfile	İmaj	Not
<code>src/main/docker/Dockerfile.jvm</code>	<code>backend</code>	Tek aşama, Quarkus JVM modu
<code>src/main/docker/Dockerfile.worker</code>	<code>video-worker</code>	Backend ile aynı jar , üstüne ffmpeg eklenmiş
<code>src/main/docker/Dockerfile.mediamtx</code>	<code>mediamtx</code>	—

job	metrics_path	Not
dcgm-exporter	/metrics	GPU donanım metrikleri
mediamtx	:9998/metrics	MediaMTX'in native metrik dinleyicisi
postgres , keycloak- postgres , redis	—	Ayrı exporter container'ları üzerinden (uygulama kendi metriğini vermiyor)
minio	/minio/v2/metrics/cluster	MINIO_PROMETHEUS_AUTH_TYPE=public ile kimlik doğrulamasız açılmış
keycloak	:9000/metrics	Doğrulanmadı — yapılandırma yorumunda açıkça "port doğrulanmadı, Keycloak 25'te ayrı bir yönetim arayüzünde olabilir" notu var
prometheus	—	Kendi kendini tarıyor

Grafana — otomatik provisioning, 9 dashboard: provisioning/datasources/ : Prometheus (varsayılan) + Loki , ikisi de access: proxy , editable: false (arayüzden değiştirilemez, tek doğruluk kaynağı bu dosyalar). provisioning/dashboards/dashboards.yml : updateIntervalSeconds: 30 — yeni bir dashboard JSON'u eklemek Grafana'yı yeniden başlatmayı gerektiriyor. 9 dashboard: altyazi-kuyruk.json , backend-genel.json , mediamtx.json , minio.json , oynatma-sagligi.json , postgres.json , redis.json , servis-durumu.json , triton-metrikleri.json .

Loki + Promtail — 7 günlük operasyonel görünürlük, denetim arşivi DEĞİL: loki-config.yaml : tek node, dosya sistemi depolama (tsdb + filesystem), retention_period: 168h (7 gün) — yorumda açıkça "bu bir denetim/arşiv sistemi DEĞİL, operasyonel görünürlük aracı" diye sınırlanmış.

promtail-config.yaml : docker.sock 'a dokunan TEK bileşen (salt-okunur mount). Kritik ayırım — docker_sd_configs yalnızca etiketleme yapıyor (hangi container hangi isimde); gerçek log içeriği Docker'ın zaten diske yazdığı /var/lib/docker/containers/*/*-json.log dosyalarından okunuyor. Backend bu dosyalara hiç dokunmuyor, yalnızca Loki'nin HTTP API'sine (LokiClient.java) sorgu atıyor. pipeline_stages: docker: {} Docker'ın json-file zarfını ({"log":"...", "stream":"...", "time":"..."}) çözüyor — bu olmadan SistemLogYorumlayici 'nin kuralları asıl log metnine değil ham zarfa bakardı. Yapılandırma dosyasında açık bir uyarı var: json-file log sürücüsü kullanıldığı **doğrulanmadı** (Docker varsayılanı, bu projede ezilmemiş).

dcgm-exporter: İmaj nvcr.io/nvidia/k8s/dcgm-exporter:3.3.9-3.6.1-ubuntu22.04 , runtime: \${CONTAINER_RUNTIME:-runc} + cap_add: SYS_ADMIN gerektiriyor (NVIDIA DCGM'in donanım sayaçlarına erişimi için).

Backend/video-worker custom metrikleri (quarkus-micrometer-registry-prometheus , application.properties 'te özel bir ayar yok, eklenti varlığı /q/metrics 'i otomatik açıyor):

Metrik	Tanımlandığı yer	Ne ölçüyor
altyazi_aktif_kanal	VadService.java	O an altyazı üretilen kanal sayısı
altyazi_kuyruk_derinlik	VadService.java	Kanal başına bekleyen bölüt sayısı
altyazi_bolut_dusme_toplam	VadService.java	Kuyruk dolduğunda/bölüt yaşı aştığında atılan bölüt sayacı
altyazi_gecikme_ms , altyazi_kapsama_yuzde	SubtitleLagMetrics.java	Üretim gecikmesi (ort/p50/p95/en kötü) ve bütçe içinde yayınlanma yüzdesi, kanal başına
altyazi_ceviri_gecikme_ms	SubtitleLagMetrics.java	Pivot yayınından hedef dilin yayınına kadar geçen ek süre, kanal+dil başına — dil etiketi tamamen dinamik, sabit bir dil listesi yok

İzleme ve Admin Panel — Derinlemesine

Admin paneli (/yonetim/*)

Giriş noktası: AppLayout 'ta yalnızca **Yönetici** rolüne görünen "Yönetim Paneline Git" düğmesi. Ayrı bir kabuk (AdminLayout) — izleme uygulamasının oynatıcıları/rehberli turu burada yok. Dört sekme:

Genel Bakış (/yonetim/genel-bakis) — üç bölüm:

- **Canlı Durum:** eşzamanlı izleyici/dinleyici (`ViewerPresence` , Redis), aktif DVR kaydı sayısı (Postgres `ActiveRecording.count()`), anlık trafik (MediaMTX `pathStates()` 'in **iki örnekleme arasındaki farkından** Mbps'e çevrilmiş hali — ilk çağrıda kıyaslanacak önceki örnek yoksa ya da MediaMTX yeniden başlayıp sayaçlar sıfırlanmışsa uydurma bir sayı üretmek yerine `null` "ölçülüyor" döner).
- **Sistem Sağlığı:** 7 bileşen (Veritabanı, Yayınlar, MediaMTX, Depolama/MinIO, Yapay Zekâ/Triton, Keycloak, Redis), her biri **bağımsız** try/catch içinde kontrol ediliyor — biri çökerse diğerlerinin sonucu etkilenmiyor.
- **Son Etkinlikler:** `etkinlik_kayitlari` tablosunun en yeni 10 kaydı.
- **Servis Metrikleri** alt bölümündeki Triton model gecikme kartları (`Whisper` , `Almanca çeviri` , `Rusça çeviri` vb.) **dinamiktir** — `STT_TARGET_LANGS` 'a göre `altyaziDilleriOku()` + `dilAdi()` (bkz. `subtitleLangs()` ile aynı kaynak) üzerinden türetiliyor; dil listesi hiçbir yerde sabit kodlanmadığı için yeni bir dil eklendiğinde panel otomatik yansıtıyor.
- Sayfa, `AdminSistemLoglarPage` ile aynı desende 15 saniyede bir otomatik tazeleniyor.

Kullanıcılar (`/yonetim/kullanicilar`) — Keycloak + yerel `AppUser` tablosunun birleşik görünümü: liste, rol değiştirme, şifre sıfırlama, silme, Keycloak ↔ yerel eşitleme. Kullanıcı adına tıklayınca açılan aktivite dialog'u panelin en ayrıntılı ekranı — yüklenen video/klip sayısı, toplam izleme süresi, en çok izlenen kanal/radyo, klip aldığı kanallar, manuel kayıt/geri sarma sayısı, son giriş — hepsi tek bir uçtan (`GET /api/admin/analitik/kullanicilar/{keycloakId}`), `etkinlik_kayitlari` tablosunun `detay` JSONB kolonundan native SQL agregasyonu ile türetiliyor, **ayrı bir izleme noktası eklenmedi**.

Etkinlikler (`/yonetim/etkinlikler`) — `etkinlik_kayitlari` 'nın ham, sayfalanmış (50'şer satır), tür/kullanıcı adına göre filtrelenebilir görünümü. Şu an loglanan olay türleri: giriş/çıkış, kanal/radyo izleme oturumları, altyazı dili değişimi, kalite değişimi, DVR geri sarma, klip oluşturma, manuel kayıt başlama/durma, kanal/radyo/kullanıcı/video CRUD'u, video oynatma oturumları (dilim ziyaretleri, tamamlandı mı, duraklama/sarma sayısı), oynatma hatası/takılma.

Analitik (`/yonetim/analitik`) — altı bölüm, sayfa açılışında tek seferde yüklenir: Canlı Sistem Durumu (Genel Bakış'la aynı 4 kutu), İçerik & Kanal Performansı (üç top-10 listesi), Depolama ve DVR Analitiği, Teknik & Hata Takibi (yayın kopma oranı dahil), Genel Kullanıcı Aktivitesi (DAU/MAU, yoğun saat), Video Tamamlanma Oranı & Isı Haritası (video başına 10 dilimlik, el yapımı bar chart — kütüphane yok).

Bu ekranların tümünde geçerli tek ortak kural: **ölçülmeden sayı verilmiyor**. Enstrümantasyonu olmayan/hesaplanamayan bir alan backend'den `null` döner; frontend'deki `ölçüm` bileşeni bunu sessizce `0` göstermek yerine "ölçülüyor" yazısıyla ayırt eder — gerçek bir ölçüm ile "henüz hiç olay olmadı" arasındaki fark bilinçli olarak korunuyor.

Tüm admin paneli **yeni bir izleme noktası eklemeden**, mevcut `etkinlik_kayitlari` tablosundan ve canlı servis sorgularından (Redis/MediaMTX/Keycloak/Triton/MinIO) türetiliyor.

Analitik dashboard'un uygulama durumu: Kullanıcı beş modüllük geniş bir analitik yüzeyi istedi, kapsam büyük olduğu için **aşamalı** ilerlendi. **Faz 1 uygulandı** — yukarıdaki altı bölüm, `AdminAnalitikResource` (`/api/admin/analitik/{ozet,icerik-performansi,depolama,teknik,genel,...}`) üzerinden. **Faz 2 planlandı, henüz uygulanmadı:** VOD tamamlama oranı + gerçek izleme ısı haritası şu an kaba (10 dilim, ziyaret var/yok) — ince taneli playback event dinleyicisi (`timeupdate` / `pause` / `seeking`) yok; oynatma hatası/buffering için `HlsPlayer.tsx` yalnızca FATAL hatalarda yeniden kuruyor, `BUFFER_STALLED` gibi fatal-olmayan olaylar dinlenmiyor; MediaMTX bant genişliği/trafik metriği Prometheus'a henüz bağlanmadı; cihaz/coğrafya kapsam dışı bırakıldı (bilinçli karar — bu depoda yalnızca web istemcisi var, GeoIP ayrı bir bağımlılık/gizlilik kararı gerektirir).

Grafana dashboard'ları (`localhost:3001`)

Tümü `src/main/docker/grafana/provisioning/dashboards/*.json` altında, otomatik yükleniyor:

Dashboard	Ne gösteriyor
Servis Durumu	Prometheus'un taradığı 11 hedeften hangisi cevap veriyor (<code>up{job="..."}</code>) — scrape-seviyesinde, admin panelin uygulama-seviyesi sağlık kontrolünden farklı bir katman
Altyazı Kuyruğu	En ayrıntılı dashboard — canlı altyazı boru hattının (ffmpeg → VAD → kuyruk → Whisper → çeviri → Redis → WebSocket) her aşaması: aktif kanal, kuyruk derinliği/büyüme, düşme oranı ve sebebi (<code>yas</code> = bütçe aşıldı, <code>kapasite</code> = kuyruk doluydu), üretim gecikmesi (ort./p50/p95, bütçe çizgisiyle), kapsama %, dil başına ek çeviri gecikmesi (<code>STT_TARGET_LANGS</code> 'taki her dil için otomatik, sabit değil), GPU kullanımı/VRAM
Oynatma Sağlığı	HLS oynatma hatası/takılma sayaçları — toplam, en çok hata veren yayınlar (top 10), kaynak türüne göre zaman serisi
Triton Metrikleri	Model başına başarılı/başarısız istek oranı, ortalama gecikme, kuyrukta bekleme süresi, hesaplama süresi, GPU bellek/kullanım

Dashboard	Ne gösteriyor
Backend & Video-Worker Genel	Aynı jar, <code>job</code> etiketiyle ayrılan iki instance — jenerik HTTP/JVM sağlığı (istek oranı, p95 gecikme, heap, GC, thread sayısı, Agroal DB havuzu)
MediaMTX	Aktif path/HLS muxer/RTSP-RTMP bağlantı sayısı (metrik adları versiyona göre değişebilir, birebir doğrulanmadı)
Postgres	İki instance (uygulama + Keycloak), <code>job</code> etiketiyle ayrı: bağlantı sayısı, commit/rollback oranı, DB boyutu, önbellek isabet oranı
Redis	Bağlı istemci, saniyedeki komut, bellek kullanımı, keyspace isabet oranı, tahliye edilen anahtar oranı
MinIO	Kullanılabilir/toplam kapasite, S3 istek oranı, disk online/offline, S3 trafiği

Henüz eklenmedi: Keycloak dashboard'u (metrik portu doğrulanmadı), alarm kuralları/Alertmanager (kapsam dışı, yalnızca görselleştirme istendi). Grafana **operasyonel/teknik derinlik** (zaman serisi, PromQL) için; admin panelin "Sistem Sağlığı" ve "Servis Metrikleri" bölümleri bunun özetlenmiş, tek-bakışlık hali.

Sistem Logları ekranı (/yonetim/sistem-loglari)

Tüm konteynerlerin loglarını, teknik olmayan birinin de anlayacağı Türkçe mesajlara çevirerek gösterir:

```
tüm container'lar → (docker.sock, salt-okunur) → Promtail → Loki
→ backend (LogQL sorgusu) → Türkçe yorum → admin panel
```

Promtail docker.sock'a dokunan **tek** bileşen (salt-okunur mount); backend hiçbir zaman docker.sock'a dokunmuyor, yalnızca Loki'nin HTTP API'sine sorgu atıyor (`LokiClient.java` , `PrometheusClient.java` ile aynı desen). Loki 7 gün saklıyor — bir denetim arşivi değil, operasyonel görünürlük aracı.

"Anlamlı" olmanın şartı — **rutin gürültü süzülüyor** (`SistemLogYorumlayici.java`): önce bilinen bir kurala (regex → Türkçe şablon) uyup uymadığına bakılır; uymuyorsa ve log yapılandırılmışsa (backend/video-worker JSON logu) kendi `level` alanına bakılır (`ERROR` → Hata, `WARN` → Uyarı, `INFO` / `DEBUG` **hiç gösterilmez**); JSON olmayan loglarda (mediamtx/postgres/redis/keycloak/minio) yalnızca `error` / `fatal` / `exception` / `panic` geçen satırlar Hata olarak gösterilir, geri kalanı hiç dönmez. Örnek tanınan kurallar: Triton CUDA OOM, bir bölütün başarıyla altyazıya çevrilmesi (Başarı), kanal kuyruğunun dolup en eski bölütün atılması (Uyarı), Redis pub/sub aboneliğinin henüz açılmamış olması (Bilgi — bilinen zararsız durum). Liste kapsamlı değil, genişletilebilir tasarlandı — yeni bir kural eklemek **KURALLAR** listesine tek satır.

Ekranda: seviye filtresi, servis adına göre arama, her satırda zaman/servis/renkli seviye rozeti/Türkçe mesaj, tıklayınca açılan katlanır ham log, 15 saniyede bir otomatik tazeleme. Uç: `GET /api/admin/sistem-loglari?servis=&seviye=&limit=200` , yalnızca Yönetici rolü.

7. Test ve Kalite Güvence

Test stratejisi

Yalnızca **backend'de** birim testleri var; frontend'de ve uçtan uca (E2E) test **bulunmuyor** (`frontend/yayin-frontend/package.json` 'da bir `test` script'i yok, `*.test.tsx` / `*.spec.tsx` dosyası yok — doğrulama şu an manuel: değişiklik sonrası tarayıcıda gerçek akış izleniyor).

Backend: **JUnit 5** (`quarkus-junit`) + **REST Assured** (entegrasyon/endpoint testleri için, `pom.xml` 'de bağımlılık olarak var). Mockito **kullanılmıyor**, Testcontainers **kullanılmıyor** — mevcut testler saf birim testi (dış bağımlılık yok, network/DB'ye çıkmıyor).

`src/test/java/org/example/` altındaki 5 test sınıfı:

Sınıf	Modül	Test sayısı
<code>SesKodekAyrıştırmaTest</code>	<code>dvr</code>	4
<code>SegmentStreamTest</code>	<code>dvr</code>	10

Sınıf	Modül	Test sayısı
SubtitleLagMetricsTest	subtitle	11
SileroVadTest	VAD	5
SpeechSegmenterTest	VAD	10

Not: `SegmentStreamTest` 'teki üç yeni "kesme" testi henüz **koşulmadı** — DVR/klip özelliği yazıldı ama kapsamlı doğrulaması yapılmadı.

Test komutları

```
./mvnw test          # tüm birim testlerini çalıştırır
./mvnw test -Dtest=SegmentStreamTest # tek sınıf
./mvnw -q -o compile -DskipTests     # yalnızca derleme (hızlı doğrulama)
```

Frontend değişikliklerinin tek gerçek doğrulama yolu şu an `docker compose build frontend` (`tsc -b && vite build` container içinde çalışıyor) — host'ta Node/npm kurulu değilse bu ZORUNLU adım.

8. Ölçeklenme Planı ve Yük Testi

Ölçekleme hedefi ve darboğaz haritası

Kapsam tanımı: "Kullanıcı" = eşzamanlı **izleyici**, kanal sayısından bağımsız bir eksen. 100 izleyici 5 kanalı mı yoksa 80 farklı kanalı mı izliyor — yük tamamen farklı, bu yüzden plan bu ikisini ayrı ayrı ele alıyor.

İki eksen birbirinden bağımsız ölçekleniyor:



Tespit edilen darboğaz sıralaması (risk yüksekten düşüğe):

Bileşen	Risk	Hangi eksene bağlı
Ağ çıkışı (egress) bant genişliği	En yüksek, en kesin	İzleyici sayısına

Bileşen	Risk	Hangi eksene bağlı
NVENC eşzamanlı encode oturumu (GeForce sürücü kilidi, tipik 3-8)	Yüksek	Eşzamanlı farklı kanal+rendition kombinasyonuna
Triton (Whisper+Marian) GPU kapasitesi	Orta-yüksek	Kanal sayısına, izleyici sayısına DEĞİL
MediaMTX bağlantı/dosya tanıtıcısı	Düşük-orta	İzleyici sayısına
Backend WebSocket + Redis pub/sub	Düşük	İzleyici sayısına, hafif
PostgreSQL / MinIO okuma yükü	Düşük	İkisine de, hafif

Egress hesabı: `gereken_egress = eşzamanlı_izleyici × ortalama_bitrate` — 100×3 Mbps (örnek değer, bu sistemde ölçülmedi) = 300 Mbps, tipik bir bağlantının çok üstünde. Azaltma sırası: düşük bitrate rendition'ı öne çıkar → HLS segmentlerini cache'le (origin yükünü azaltır, **toplam egress'i azaltmaz**) → bağlantı kapasitesini büyüt.

NVENC tarafında kritik ayırım: **aynı kanalı izleyen 100 kişi tek encode oturumu paylaşır** (HLS ile çoğaltılıyor) — risk kullanıcı sayısından değil, eşzamanlı izlenen **farklı kanal çeşitliliğinden** geliyor. Yazılım kodlamaya düşmenin CPU maliyeti ölçüldü: donanımda kanal başına %14, yazılımda **%142 CPU** — 40 kanal yazılımda ~57 çekirdek gerektirir, pratik değil.

Ürün kararı: altyazı her zaman mı, yalnızca izlenen mi

Planın en kritik çelişkisi şuydu: GPU verimliliği için `VadService` 'in yalnızca izlenen kanalları çözümlemesi gerekiyordu, ama DVR/klip'in her zaman altyazılı olabilmesi için tam tersi — her aktif kanalın sürekli işlenmesi — gerekiyordu. İkisi aynı anda doğru olamaz.

Karar verildi: her zaman tüm aktif kanalları işle, `VadService` izleyici bazlı filtrelenmeyecek. Gerekçe: klip alımına altyazı dili tercihi eklenecek ve aynı özellik yüklenen videolara da genişletilecek — ikisi de kanalın sürekli işlenmiş, altyazısı hazır beklemesini gerektiriyor. GPU israfı (izlenmeyen kanalın da işlenmesi) bilinçli kabul edildi. Rendition (kalite dönüştürme) ise bu karardan bağımsız — talebe bağlı üretilebilir, çünkü rendition NVENC/CPU kodlama kullanıyor, altyazı Triton/Whisper; paylaşımlı kaynak yok.

Bu kararın **doğrudan sonucu**: GPU yükü hiçbir zaman "gerçekten izlenen kanal sayısı"na düşmeyecek, her zaman "toplam aktif kanal sayısı"na bağlı kalacak — 100 kullanıcı az sayıda popüler kanalda toplansa bile kazanç yok.

İlerleme durumu

İş	Durum
Ürün kararı (altyazı her zaman / rendition talebe bağlı)	✅ Verildi
<code>VadService</code> 'i izlenen kanala göre filtrele	❌ Yapılmayacak — yukarıdaki kararın doğrudan sonucu
Rendition'ı yalnızca izlenirken üret	✅ Uygulandı — <code>TranscodeCommand.buildOnDemand</code> , MediaMTX'in native <code>runOnDemand</code> / <code>runOnDemandCloseAfter</code> ilkesiyle; her rendition kendi bağımsız ffmpeg sürecini alıyor, path'in okuyucu sayısına göre MediaMTX'in kendisi başlatıp durduruyor (<code>channels.rendition-start-timeout=15s</code> , <code>channels.rendition-close-after=60s</code>). Soğuk başlangıç gecikmesi ve zapping churn'ü ölçülmedi
HLS önüne nginx cache	✅ Uygulandı — <code>.m3u8</code> 2sn TTL, <code>.ts</code> / <code>.m4s</code> 24 saat immutable, <code>proxy_cache_lock on</code> (thundering-herd önleme)
<code>altyazilar</code> tablosuna retention/süpürme	✅ Uygulandı — <code>storage.subtitle-retention</code> (varsayılan kapalı), <code>RetentionSweeper</code> 'a dördüncü iş olarak eklendi, <code>V26__altyazi_baslangic_indeksi.sql</code> ile tam tablo taraması önlendi
Gözlemlenebilirlik (Prometheus+Grafana tüm servisler)	✅ Uygulandı
100 sahte istemciyle yük testi	✅ Koşuldu (aşağıda)
Klip + yüklenen video altyazı tercihi (WebVTT export)	Henüz başlanmadı — video tarafı, canlı kanal segment bazlı STT boru hattından ayrı yeni bir alt sistem gerektiriyor

İş	Durum
Bant genişliğini büyüt	Bekliyor — işletme kararı, kod değişikliği değil
Üretim GPU'suna geç (Triton)	Ölçüldü — bkz. "Tur 3" aşağıda: CTranslate2 sonrası 98 kanalda hâlâ OOM yok
Quadro/RTX Ada/Tesla'ya geç (NVENC kilidi yok)	Bekliyor — rendition-talebe-bağlı + yük testi sonuçları netleşmeden gerekip gerekmediği bilinmiyor

Yük testi sonuçları (ilk gerçek koşum)

Tur 1 — 100 VU, tek kanal (`loadtest/canli-izleme-senaryosu.js` , k6, hedef: `dw-ar` , 60 saniye):

```
checks_total: 2046  başarı: %100  hata: %0
http_req_duration: ort=121.44ms p50=585.78µs p90=356.52ms p95=852.84ms max=3s
http_req_failed: %0 (0/2869)
ws_connecting: ort=6.78ms p95=14.48ms
ws_session_duration: 100 oturumun tamamı tam 60sn (kesilme yok)
```

Tur 2 — 100 VU, 15 farklı kanala dağıtılmış (`loadtest/canli-izleme-cok-kanal-senaryosu.js` , round-robin):

```
checks_total: 2052  başarı: %100  hata: %0
http_req_duration: ort=120.8ms p95=575.8ms max=4s
GPU bellek – test ÖNCESİ: 5767 MiB / 6141 MiB
GPU bellek – test SONRASI: 5767 MiB / 6141 MiB (BİREBİR AYNİ)
```

Sonuç: Backend/MediaMTX/WebSocket katmanı, izleyici tek kanalda ister 15 kanala dağılsın, **%0 hata** — bu katmanlarda mevcut bir darboğaz yok. Kritik doğrulama: GPU belleği test öncesi/sonrası birebir aynı kaldı — 100 izleyicinin 15 kanala dağılması Triton'a **sıfır ek yük** bindirdi, "yük kanal sayısına bağlı, izleyici sayısına değil" varsayımı artık **ölçülerek** doğrulandı.

Tur 3 — 100 kullanıcı, 100 kanal (`loadtest/canli-izleme-100-kanal-senaryosu.js` , k6, CTranslate2 sonrası — Triton/GPU'nun gerçek tavanını **kanal sayısı** ekseninde ölçmek için, Tur 1/2'nin izleyici-ekseninden farklı olarak):

Kanal sayısını 100'e çıkarmak için iki bağımsız uygulama sınırının (`channels.max-active` varsayılan 16, `vad.max-channels` varsayılan 20 — bkz. §5 "channel" ve "VAD" paketleri) geçici olarak yükseltilmesi gerekti; 15 gerçek kaynak (TRT/DW) aynı URL'lerle ~7'şer kez kopyalanıp 100 bağımsız kanal kaydı oluşturuldu (her kopya kendi ffmpeg/VAD/Whisper/Marian tüketicisine sahip, gerçek GPU yükü üretiyor):

```
Aktif kanal: 100 (channels.max-active geçici olarak 100'e çıkarıldı)
Altyazı üreten kanal: 98 (vad.max-channels geçici olarak 100'e çıkarıldı;
    2 kanal kaynağa bağlanamadı – MediaMTX "ready"
    olmadı, uygulama sınırı değil, kaynak sorunu)
GPU bellek: 2,2-2,5 GiB / 6141 MiB, %19-100 GPU-util – 5 dakika boyunca
    İSTİKRARLI, OOM YOK (bkz. gpu_ramp2.log ölçümü)
Triton: healthy, boyunca kesintisiz
Çeviri başarı oranı (2 dakikalık pencere): 732/732 (%100)
video-worker hata sayısı (cuda/timeout/ulaşılamadı): 0
"VAD kanal sınırı dolu" uyarısı: 0 (98 ≤ 100 sınırı içinde kaldı)
Bölüt düşme (altyazi_bolut_dusme_toplam): 0

k6 – 100 VU, 60 sn, kanal listesi setup()'ta API'den çekildi:
checks: %98,79 (1973/1997)  http_req_failed: %0,85 (24/2810)
ws_sessions: 100/100 tam 60sn bağlı kaldı
```

Sonuç: CTranslate2 migrasyonundan önce ölçülen ~15-20 kanallık GPU tavanı **artık geçerli değil** — bu, ONNX Runtime'ın büyüyen VRAM ayak izinin bir sonucuydu (bkz. aşağıdaki "genel ders"), model motorunun kendisinin sınırı değildi. CTranslate2 ile 98 gerçek kanal aynı anda işlenirken GPU belleği hâlâ kartın **üçte birinin altında** kaldı — bu donanımda (RTX 4050, 6 GB) gerçek GPU tavanı

bu testte **bulunamadı** (100 kanalın altında kırılmadı). Kalan pratik sınır artık GPU değil, iki yazılım güvenlik sınırı: `channels.max-active` ve `vad.max-channels` — ikisi de varsayılan olarak çok daha düşük (16/20) ve kasıtlı olarak muhafazakâr seçilmiş, gerçek GPU tavanı değil.

Bu testin göstermediği (hâlâ ölçülmedi): Gerçek kırılma noktası — 100'ün üzerinde kaç kanalda GPU OOM'a döner. Ayrıca bu test hâlâ 15 **benzersiz** kaynağın kopyalarını kullandı; farklı ses/dil karışımına sahip gerçekten 100 farklı kaynakla sonuç değişebilir (ölçülmedi).

GPU kapasite sınırı — genel ders

Aşağıdaki ~5,3-5,8 GB tavanı ONNX Runtime dönemine ait (bkz. Tur 3'ün gösterdiği gibi CTranslate2 sonrası bu tavan artık geçerli değil). O dönem GPU tavanı model boyutuyla değil, **eşzamanlı işlenen kanal/istek sayısının ürettiği dinamik bellekle** belirleniyordu — fp16 export, daha küçük dil modelleri (en-fr, en-eo) gibi denemelerin hepsi aynı ~5,3-5,8 GB gerçek-yük tavanına yakınsamıştı. Triton'un varsayılan dosya tanıtıcısı limiti (1024) whisper+Marian stub süreçlerinin gerçek ihtiyacına yetmediği için `ulimits.nofile: {soft: 65536, hard: 65536}` olarak ayarlanmış durumda — bu, motor bağımsız, hâlâ geçerli.

Triton'un iç istek kuyruğu, istemcilerin 120sn'de vazgeçmesinden **haberdar değil** — bir yoğunluk anında biriken işler istemciler vazgeçtikten sonra da sırayla işlenmeye devam eder, yeni istekler bu kuyruğun arkasına eklenir (kanal sayısını azaltmak kuyruğu boşaltmaz). Kuyruğu sıfırlamanın tek yolu Triton konteynerini tamamen yeniden başlatmaktır (`docker compose stop triton && up -d triton`); ardından `video-worker`'ın da yeniden başlatılması gerekir (bkz. operasyonel adımlar için `docs/kullanim-kilavuzu.md` §5).

Ölçülen VRAM tüketimi (RTX 4050, 6141 MiB — tarihsel ölçüm)

Konteyner **içinden** `nvidia-smi` yanıtıcı (PID namespace izolasyonu "No running processes found" gösteriyor) — gerçek tüketim yalnızca **host'ta** `nvidia-smi --query-compute-apps` ile görünüyor. O anki yapılandırmada (`STT_MODEL=base`, 3 sabit dil, hepsi 1 instance):

```
tritonserver (ana süreç):          146 MiB
whisper stub:                     3064 MiB
marian stub (dil başına, ort.):    452-1004 MiB
                                  TOPLAM: ~5650 / 6141 MiB
```

Yalnızca modellerin **yüklenmiş olması** (henüz hiçbir istek işlenmeden) kartın %92'sini dolduruyor — geri kalan pay gerçek bir transcribe işleminin çalışma belleği için yetersiz kalıp `CUDA out of memory` ürettiyordu. Bu ölçüm sabit değil (dil sayısı/instance sayısı/model boyutu değişince tekrar ölçülmeli) ama kartın **yükleme aşamasında bile sınırdan** olduğunu somut olarak gösteriyor.

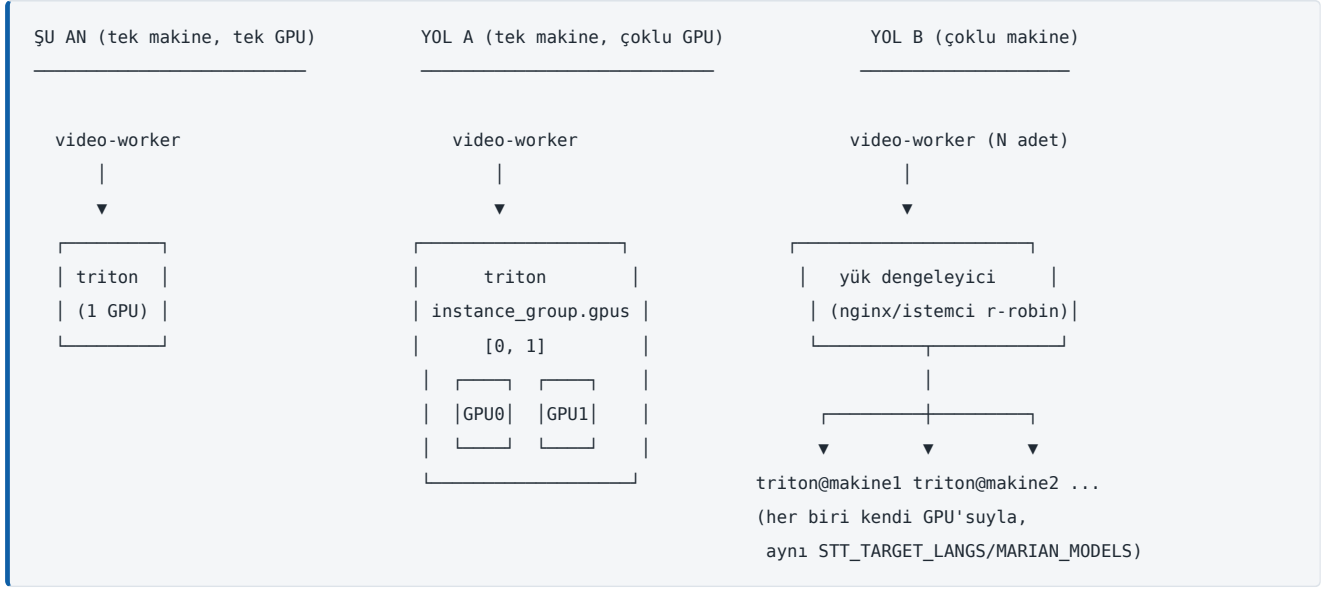
Batching — göç öncesi ölçüm (eski `stt-worker`, artık kod yok)

Bu ölçüm Triton'a geçmeden ÖNCEKİ, artık repoda olmayan Python `stt-worker`'a ait (`STT_BATCH_WINDOW_MS`, `STT_BATCH_MAX_SIZE`, elle yazılmış `BatchCoalescer`) — Triton'ın kendi `dynamic_batching`'i farklı bir mekanizma, ama ortaya çıkardığı genel örüntüler (iki-uçlu dağılım, kesit başına sabit-değil VRAM maliyeti) hâlâ öğretici, bu yüzden tarihsel referans olarak tutuluyor:

- Whisper:** ortalama batch 2,27 kesit, dağılım iki-uçlu (%57 tekil, %18 gözlenen tavan olan 6'lı) — trafiğin %74,8'i yine de birleştirilmiş bir batch'in parçasıydı.
- Kesit başına GİRDİ tensörü sabit: 0,92 MB** (Whisper her girdiyi 30 saniyelik sabit pencereye dolduruyor, kesit süresi fark etmiyor).
- Kesit başına GERÇEK VRAM maliyeti (B≥4'te sabitlendi): 80 MB/kesit, ölçüldü** — girdi tensörünün (0,92 MB) 87 katı; fark modelin ara hesaplama belleğinden (encoder/decoder, attention) geliyor.
- Bu `v` değeriyle B_maks formülü: $(V_{kart} - V_{taban}) / (N_{es} \times v)$ — o zamanki 6 GB kart için ≈8,1, ayarlanan tavanla (8) zaten örtüşüyordu.
- Çeviri:** ortalama 2,55 metin/batch, girdi belleği ihmal edilebilir (token ID'leri, ses matrisi değil) — ölçülen kazanç (5,93×) Whisper'dan (1,97×) yüksek çıktı çünkü paylaşılabilecek veri maliyeti neredeyse sıfır, kazancın tamamı sabit-maliyet (kernel açma) paylaşımından geliyor.
- Bu ölçüm sırasında ayrıca `altyazi_bolut_dusme_toplam` Grafana panellerinin (Micrometer'in eklediği `_total` soneki sorguda eksikti) baştan beri boş/yanıtlı döndüğü bulunup düzeltildi — gerçek düşme oranının ~0,22 bölüt/sn olduğu ortaya çıktı.

Yatay ölçekleme — plan, HENÜZ UYGULANMADI

Bu makinede hâlâ tek GPU var; aşağıdakiler donanım eklendiğinde uygulanacak bir plan, şu an çalıştırılabilecek bir komut değil.



Kilit gerçek: Whisper ve Marian modelleri **durumsuz** (stateless) — `sequence_batching` hiçbirinde yok, her istek bağımsız işleniyor. Yani N tane birebir aynı Triton kopyası çalıştırılırsa herhangi bir istek herhangi bir kopyaya gidebilir (session affinity gerekmiyor) — klasik "stateless servisi yatay ölçekle" problemi, en kolay türden. Tek istisna: Marian'ın `response_cache`'i **kopyaya özel** (yerel), N kopya arasında round-robin yapılırsa isabet oranı düşebilir (hata değil, verimlilik kaybı).

Yol A — aynı makineye ikinci GPU (önce bu denenmeli, en ucuz): Triton tek süreçte birden fazla GPU yönetebiliyor (`instance_group.gpus: [0,1]`). Gerekli: `docker-compose.yaml`'a açıkça `NVIDIA_VISIBLE_DEVICES` eklemek (şu an yalnızca `runtime: nvidia` var, sürücüyü göre belirsiz davranabilir) + `config.pbtxt`'lerde GPU ID'lerini genişletmek. **Yeni container/imağ/kod gerekmez**, yalnızca donanım + iki config satırı. Sınır: tek makinenin fiziksel GPU yuvası/güç/PCIe kapasitesi.

Yol B — birden fazla makineye Triton dağıtmak (gerçek yatay ölçekleme):

- İmağ dağıtımı** — şu an `yayın/triton` yalnızca yerel build ediliyor, registry'ye push edilmiyor; birden fazla makine için bir registry (aksi halde her makine kendi build'ini yapar, sürüm tutarsızlığı riski).
- Aynı yapılandırma** — her kopya aynı `STT_TARGET_LANGS / MARIAN_MODELS` ile build edilmeli (farklı build-arg = makineden makineye farklı kalitede/dilde çeviri, sessiz bir tutarsızlık); instance sayıları (`WHISPER_INSTANCES` vb.) her makinenin kendi VRAM'ine göre ayrı olabilir.
- Yük dengeleyici — asıl eksik parça** — Triton kendisi çoklu-sunucu dengelemesi yapmıyor. İki seçenek: ayrı bir reverse proxy (nginx `upstream` /HAProxy, `/v2/health/ready` ile aktif sağlık kontrolü) ya da istemci tarafı basit round-robin (`TritonClient.java`'ya virgülle ayrılmış `TRITON_URLS` + periyodik `saglikliMi()` ile sağlıksız geçici çıkarma). Dikkat: `strict_readiness=1` yüzünden tek bir model bile yüklenemezse Triton kendini TAMAMEN "not ready" sayıyor — yük dengeleyici bunu doğru okumalı.
- `TritonClient.saglikliMi()` güncellenmeli** — şu an TEK Triton'un `/v2/health/ready` 'sine bakıyor; N kopyalı kurulumda admin panelin "Sistem Sağlığı" kartı toplu bir görünüme ("kaçı sağlıklı") dönmeli.
- Prometheus scrape config'e yeni hedefler** — mevcut PromQL sorguları `instance` etiketine göre zaten otomatik topladığı için sorgu tarafında değişiklik gerekmiyor.
- `ulimits.nofile: 65536` her yeni kopyaya taşınmalı** — Triton'ın varsayılan 1024 FD limitinin tükenip `accept()` çöktüğü ölçüldü; kopyalanan deployment tanımında unutulursa o kopya sessizce çökebilir.

Sıra: önce Yol A tüketilmeli (donanım varsa), yetmeyince Yol B'ye (önce istemci tarafı round-robin, ihtiyaç büyürse ayrı proxy). Kapsam dışı: kaç kopya/GPU gerektiği (gerçek donanım olmadan ölçülemez, yalnızca tahmin edilebilir) ve Kubernetes/Nomad gibi bir orkestrasyon katmanı (bu repo düz Docker Compose kullanıyor).

Diğer belgeler

Belge	İçerik
<code>README.md</code>	Hızlı kurulum, <code>.env</code> alan referansı, altyazı bütçe/gecikme özeti
<code>docs/kullanım-kilavuzu.md</code>	Kurulum ve günlük kullanım kılavuzu — sistem yöneticileri/son kullanıcılar için